CLOUD-BASED HOSPITAL MANAGEMENT SYSTEM WITH AI DIAGNOSIS

A Project Report for Industrial Training and Internship

Submitted by

SUPRIYO NASKAR

In the partial fulfillment of the award of the degree of

B.TECH

Of

Future Institute of Engineering and Management

Ardent Computech Pvt.Ltd.

CERTIFICATE FROM SUPERVISOR

This is to certify that**, "Supriyo Naskar"** have completed the project titled "CLOUD BASED HOSPITAL MANAGEMENT SYSTEM WITH AI DIAGNOSIS under my supervision during the period from **07/01/2026** to **13/03/2026** which is in partial fulfillment of requirements for the award of the **CSE** degree and submitted to the Department of **CSE** of **Future Institute Of Engineering And Management, Sonarpur"**.

Signature of the Supervisor Date:

Name of the Project Supervisor:

BONAFIDE CERTIFICATE

Certified that this project work was carried out under my supervision Smart Job Matching and Recruitment System is the bonafide work of

Name of the student: SUPRIYO NASKAR ***Signature: Supriyo Naskar***

Signature

Name: MR. SUBHAJIT SANTRA

PROJECT MENTOR

SIGNATURE

Name: EXAMINERS

Ardent Original Seal

ACKNOWLEDGEMENT

The achievement that is associated with the successful completion of any task would be incomplete without mentioning the names of those people whose endless cooperation made it possible. Their constant guidance and encouragement made all our efforts successful.

We take this opportunity to express our deep gratitude towards our project mentor,

Subhajit Santra for giving such valuable suggestions, guidance and encouragement during the development of this project work.

Last but not the least we are grateful to all the faculty members of **Ardent Computech Pvt. Ltd.** for their support

Content Page

1. Company Profile ---------- 1
2. Introduction ---------- (2-4)
   i. *Objective* ---------- *3*
   ii. *Scope of the project* ---------- *4*
3. System Analysis ---------- (5 - 10)
   i. *Identification of Need* ---------- *5*
   ii. *Feasibility Study* ---------- *5*
   iii. *Workflow / Design Model* ---------- *5*
   iv. *Study of Current System* ---------- 6-*9*
   v. *Inputs & Outputs* ---------- *9*
   vi. *Software Requirement Specification (SRS)* ---------- *10*
4. System Design ---------- (11 - 14)
   i. *Data Flow Diagram (DFD)* ---------- *12*
   ii. *Sequence Diagram* ---------- *13*
   iii. *Use Case Diagram* ---------- *14*
5. UI & Database Snapshots ---------- *(15-56)*
6. Conclusion ---------- 57
7. Future Scope & Enhancements ---------- 58
8. Bibliography ---------- 59

1. COMPANY PROFILE

ARDENT (Ardent Computech Pvt. Ltd.), formerly known as Ardent Computech Private Limited, is an ISO 9001:2015 certified Software Development and Training Company based in India. Operating independently since 2003, the organization has recently undergone a strategic merger with ARDENT Technologies, enhancing its global outreach and service offerings.

ARDENT Technologies:
ARDENT Technologies delivers high-end IT services across the UK, USA, Canada, and India. Its core competencies lie in the development of customized application software, encompassing end-to-end solutions including system analysis, design, development, implementation, and training. The company also provides expert consultancy and electronic security solutions. Its clientele spans educational institutions, entertainment companies, resorts, theme parks, the service industry, telecom operators, media, and diverse business sectors.

ARDENT Collaborations
ARDENT Collaborations, the Research, Training, and Development division of ARDENT (Ardent Computech Pvt. Ltd.), offers professional IT-enabled services and industrial training programs. These are tailored for freshers and professionals from B.Tech, M.Tech, MBA, MCA, BCA, and MSc backgrounds. ARDENT (Ardent Computech Pvt. Ltd.) provides Summer Training, Winter Training, and Industrial Training to eligible candidates. High- performing students may qualify for stipends, scholarships, and additional benefits based on performance and mentor recommendations.

Associations and Accreditations
ARDENT (Ardent Computech Pvt. Ltd.) is affiliated with the National Council of Vocational Training (NCVT) under the Directorate General of Employment & Training (DGET), Ministry of Labour & Employment, Government of India. The institution upholds strict quality standards under ISO 9001:2015 certification and is dedicated to bridging the gap between academic knowledge and industry skills through innovative training programs.

2. INTRODUCTION

The healthcare industry is undergoing a digital revolution. Traditional paper-based hospital management is being replaced by cloud-based systems that offer better efficiency and data security.

This project introduces a Cloud-Based Hospital Management System (HMS) designed to streamline hospital operations. Beyond simple record-keeping, the system uses AI to assist doctors in diagnosing diseases based on patient data. Built with modern web technologies, the platform focuses on speed, security, and a user-friendly interface for both medical staff and patient.

In addition to basic management functions such as patient registration, appointment scheduling, and medical record handling, the system incorporates an AI-based diagnosis feature. This intelligent module helps in analyzing patient symptoms and providing preliminary diagnosis support, assisting doctors in decision-making and improving overall treatment efficiency. The platform is developed using modern web technologies, focusing on user-friendly design, performance, and reliability, making healthcare services faster, smarter, and more accessible.

2. i. OBJECTIVE

The primary objective of this project is to build a scalable and interactive hospital management system that improves patient care.

The specific goals include:

- **Centralized Patient Records:** To develop a secure database where doctors can access patient history instantly.
- **Efficient Scheduling:** To create an automated system for booking and managing doctor appointments.
- **AI-Powered Diagnosis:** To implement an AI module capable of providing preliminary health assessments based on symptoms.
- **Secure Cloud Storage:** To ensure all sensitive medical data is encrypted and stored safely on the cloud using industry standards like JWT.
- **Real-Time Dashboard:** To design a dashboard that allows hospital admins to monitor bed availability and staff performance in real-time.

2.ii SCOPE of Project

The scope of this project includes the design and development of a full-stack HMS ecosystem.

Functional Scope:

- **User Management:** Secure login for Doctors, Patients, and Administrators using multi-factor authentication.
- **Appointment System:** A dynamic calendar allowing patients to find and book available slots with specific specialists.
- **AI Diagnostic Assistant:** A 24/7 AI tool powered by machine learning to help users understand symptoms before seeing a doctor.
- **Electronic Health Records (EHR):** Digital storage for prescriptions, lab reports, and medical history.

Technical Scope:

- **MERN Stack:** Using Mongo DB for the database, React for the frontend, and Node.js/Express for the backend.
- **AWS Integration:** Hosting the application and database on AWS for 99.9% uptime.
- **Secure APIs:** Implementation of JWT for secure communication between hospital departments.

3. SYSTEM ANALYSIS

3.i. IDENTIFICATION OF NEED

Modern hospitals often struggle with manual paperwork and fragmented data across different departments.

Critical needs identified:

- **Delayed Treatment:** Patients often wait too long for diagnosis due to manual record retrieval. A centralized system is needed.
- **Lack of Diagnostic Support:** Doctors need AI-driven insights to help identify patterns in complex medical data.
- **Security Concerns:** Paper records are prone to loss or unauthorized access. A hardened cloud security architecture is essential.

3. ii. FEASIBILITY STUDY

 Technical Feasibility: The MERN stack provides the modularity and speed required for healthcare applications.

 Economic Feasibility: By using open-source tools and cloud pay-as-you-go models, the project is cost-effective to build and maintain.

 Operational Feasibility: The interface is designed for simplicity, allowing medical staff with minimal IT training to use it effectively.

.

3.iii. WORKFLOW

The system follows the **Waterfall Model** for structured development:

1. **Requirements Analysis:** Defining hospital needs like patient registration and AI accuracy.
2. **System Design:** Mapping out the database schema and AI model architecture.

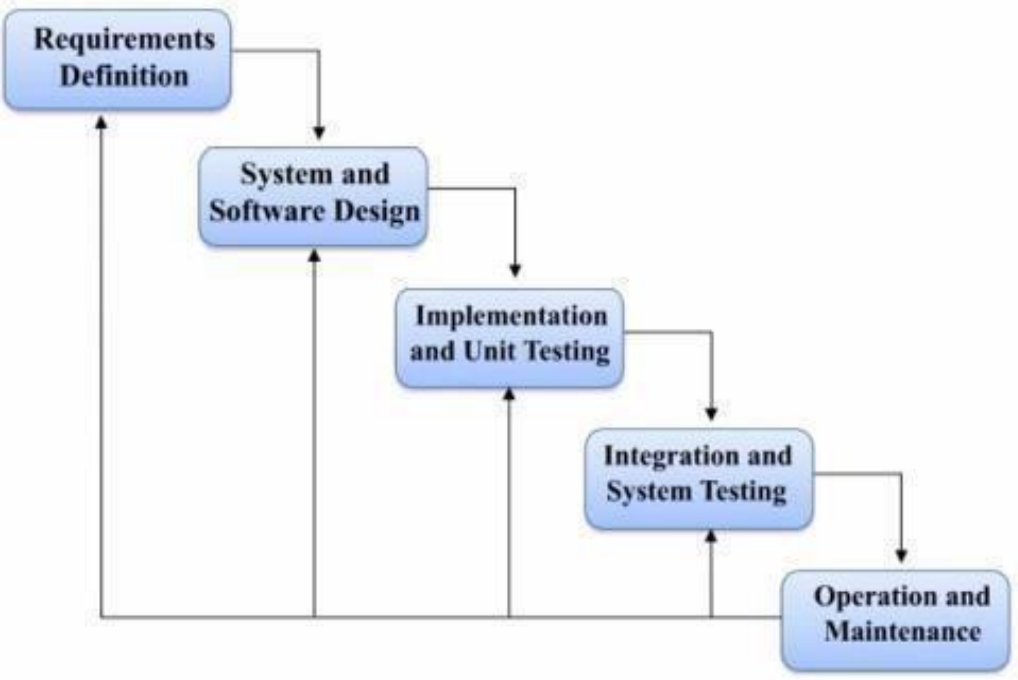
3. **Implementation:** Coding the modules with React and Node.js.
4. **Testing:** Validating the AI diagnosis accuracy and data security.

:

❖ Waterfall Model Design Phases:

- **Requirements Analysis:**
  This is the initial phase where all system requirements are clearly defined. The functionalities such as patient registration, appointment scheduling, medical record management, and AI-based diagnosis are identified. All requirements are documented in the Software Requirement Specification (SRS) before moving to the next phase.
- **System Design:**
  In this phase, the requirements are converted into a detailed system design. This includes designing Data Flow Diagrams (DFDs), database schema (Patients, Doctors, Appointments, Reports), and defining API structures. The architecture ensures smooth communication between frontend, backend, and cloud database.
- **Implementation:**
  This is the coding phase where the actual system is developed. The frontend is built using React.js, and the backend is developed using Node.js and Express.js. Database integration is done using MongoDB, and AI diagnosis features are implemented using appropriate APIs or machine learning models.
- **Testing:**
  After development, the system is tested to ensure all functionalities work correctly. This includes checking patient data handling, appointment booking, AI diagnosis accuracy, and overall system performance. Bugs and errors are identified and fixed during this phase.
- **Deployment & Maintenance:**
  In this final phase, the system is deployed on a cloud platform, making it accessible to users. Regular updates, bug fixes, and feature improvements are carried out as part of maintenance to ensure the system remains efficient and up-to-date.

- **<u>Advantages</u>:**
  - **Discipline and Structure:**
    Each phase has clear deliverables (like SRS, DFDs, and database design), making the development process organized and easy to track.
  - **Clear Milestones:**
    It is easy to identify the completion of each stage, helping developers and supervisors understand project progress.
  - **Design Stability:**
    Since the system design is finalized before coding, there is less confusion during development and better planning of hospital workflows and AI modules.
  - **Suitable for Well-Defined Projects:**
    For a hospital management system with fixed features like patient records, appointments, and diagnosis support, this model works effectively.



- **<u>Disadvantages</u>:**
  - **Inflexibility:**
    Once a phase is completed, it is difficult to make changes (for example, modifying AI diagnosis logic later).
  - **Late Testing Risk:**
    Errors in system logic (like incorrect patient data handling) may be detected only after implementation, making them harder to fix.
  - **Delayed User Feedback:**
    The working system is visible only in later stages, so early feedback from doctors or users is limited.
  - **Not Ideal for Complex AI Development:**
    AI features may require continuous testing and improvement, which is difficult in a strict Waterfall approach.

3.iv. STUDY OF THE CURRENT SYSTEM

- Hospital management software in clinics
  Manual record-keeping systems
- provide basic healthcare services but lack advanced features and proper integration.
- **Problems in Existing Systems:**
- **Lack of Centralization:**
  Data like patient records and appointments are stored separately, making access difficult.
- **Limited Accessibility:**
  Most systems are not cloud-based, so remote access is not possible.
- **No AI Support:**
  There is no intelligent system for helping in diagnosis or decision-making.
- **Manual Errors:**
  Paper-based systems often lead to mistakes and delays.
- **Security Issues:**
  Patient data is not always properly protected.

3.v. INPUT AND OUTPUT

- **Input:**

  Patient details during registration (name, age, contact, medical history)
  Doctor login credentials
  Appointment booking details
  Symptoms entered for AI diagnosis

- **Output:**

  Patient dashboard
  Appointment details and schedules
  Medical reports and history
  AI-based diagnosis suggestions

❖ Hardware & Software Requirements

To successfully develop and deploy this system, the following environment is required:

- **Hardware Requirements**

Component	Minimum Specification	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5/i7 or Apple M1/M2/M3
RAM	8 GB	16 GB
Disk Space	50 GB	100 GB SSD
Network	Broadband Internet	Stable high-speed fiber connection

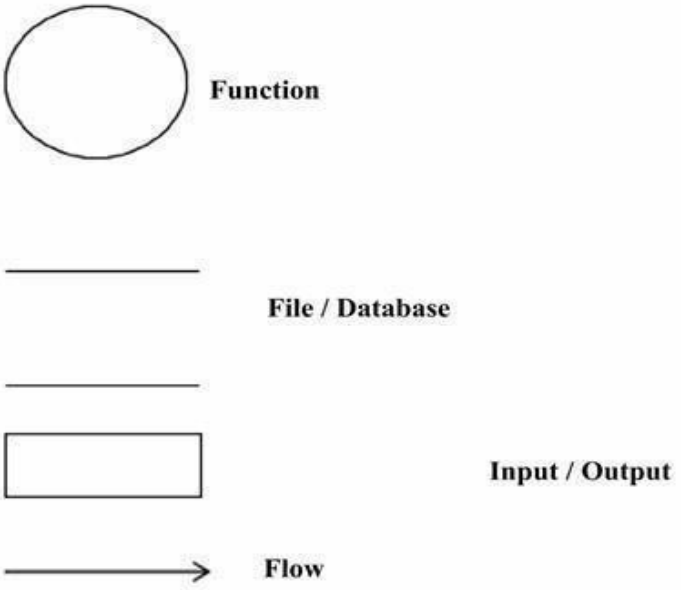
- **Software Requirements**

Category	Tool/Environment
Operating System	Windows 10/11, macOS, or Linux (Ubuntu)
Development Environment	VS Code (with MERN extensions)
Frontend Framework	React.js (v18+)
Backend Framework	Node.js (v20+) & Express.js
Database	MongoDB (v7+) & MongoDB Compass
API Testing	Postman or Insomnia
Version Control	Git & GitHub

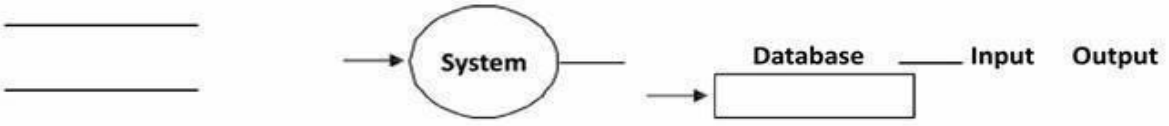
4. SYSTEM DESIGN

4.i. DATA FLOW DIAGRAM (DFD)

DFD Notation:



DFD Example:



Steps to Construct Data Flow Diagram:

Four Steps are generally used to construct a DFD.

Process should be named and referred for easy reference. Each name should be representative of the reference.

The names of data stores, sources, and destinations are written in capital letters.

Rules for constructing a Data Flow Diagram:

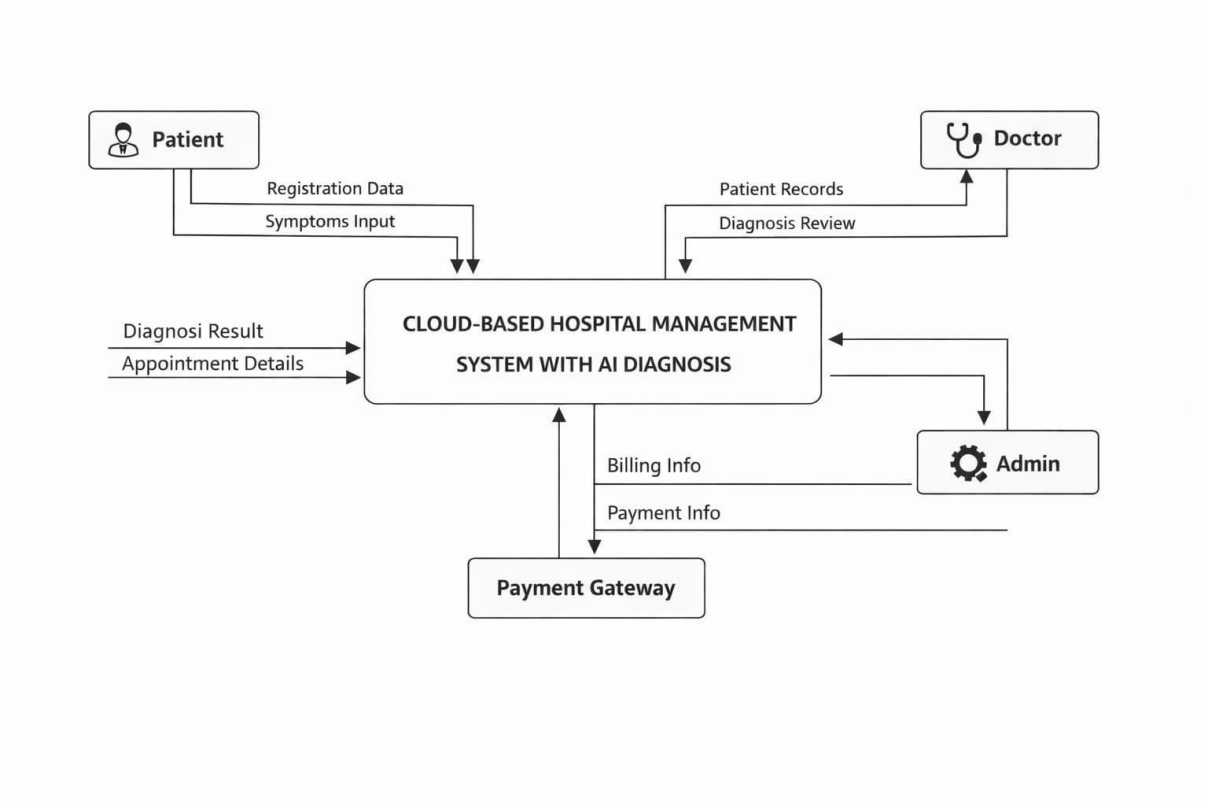
Arrows should not cross each other.

Squares, Circles, and Files must bear a name.

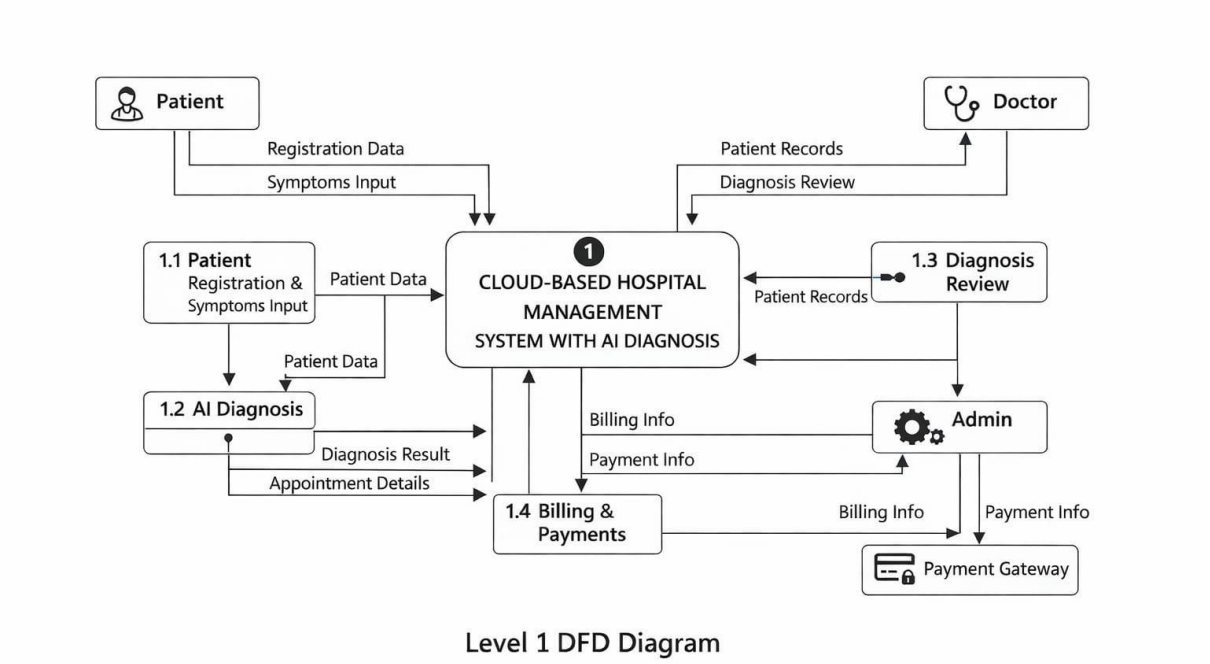
Decomposed data flow squares and circles can have the same names.

Draw all data flow around the outside of the diagram.

❖ Level 0 DFD:

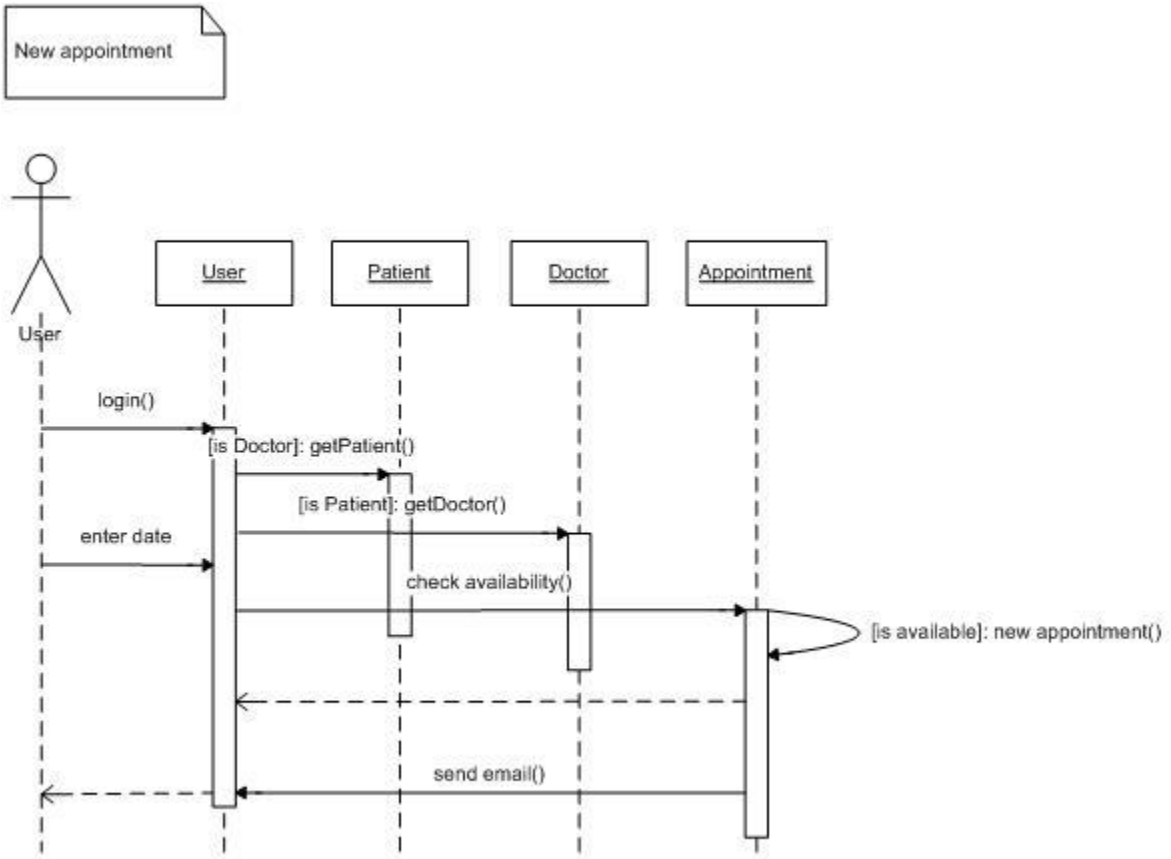


❖ LEVEL 1 DFD



Level 1 DFD Diagram

4.ii. SEQUENCE DIAGRAM



- User logs into the system to authenticate their identity.
- System identifies the user role and retrieves the corresponding Patient or Doctor profile.
- User enters the desired date for the appointment.
- System checks the doctor's availability for the specified date.
- New appointment is created if the slot is available.
- Confirmation email is sent to the user and a success response is returned.

4.iii. USE CASE DIAGRAM:

1. Actors

- **Patient:** Uses services (booking/reports).
- **Doctor:** Manages care (EHR/diagnosis).
- **Admin:** Controls system (staff/security).
- **AI Engine:** Processes data (analysis/prediction).

2. Use Cases

Patient Portal

- **Access:** Login & History.
- **Tasks:** Booking & Payments.
- **Health:** Symptom Input & AI Reports.

Clinical Operations (Doctor)

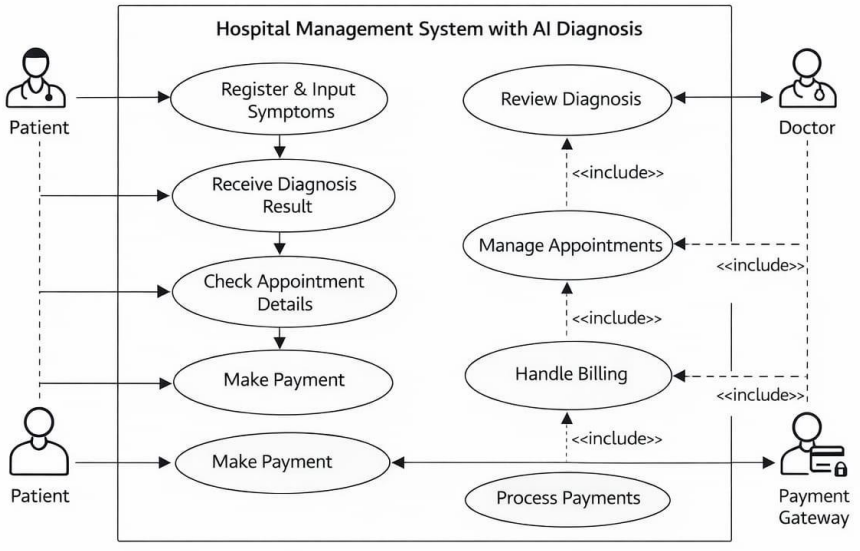
- **Records:** EHR & Lab Results.
    - **Tools:** Triggering AI Analysis.

Administration

- **Staff:** Hiring/Managing Personnel.
- **Security:** Protecting Data Records.

AI Processing (System)

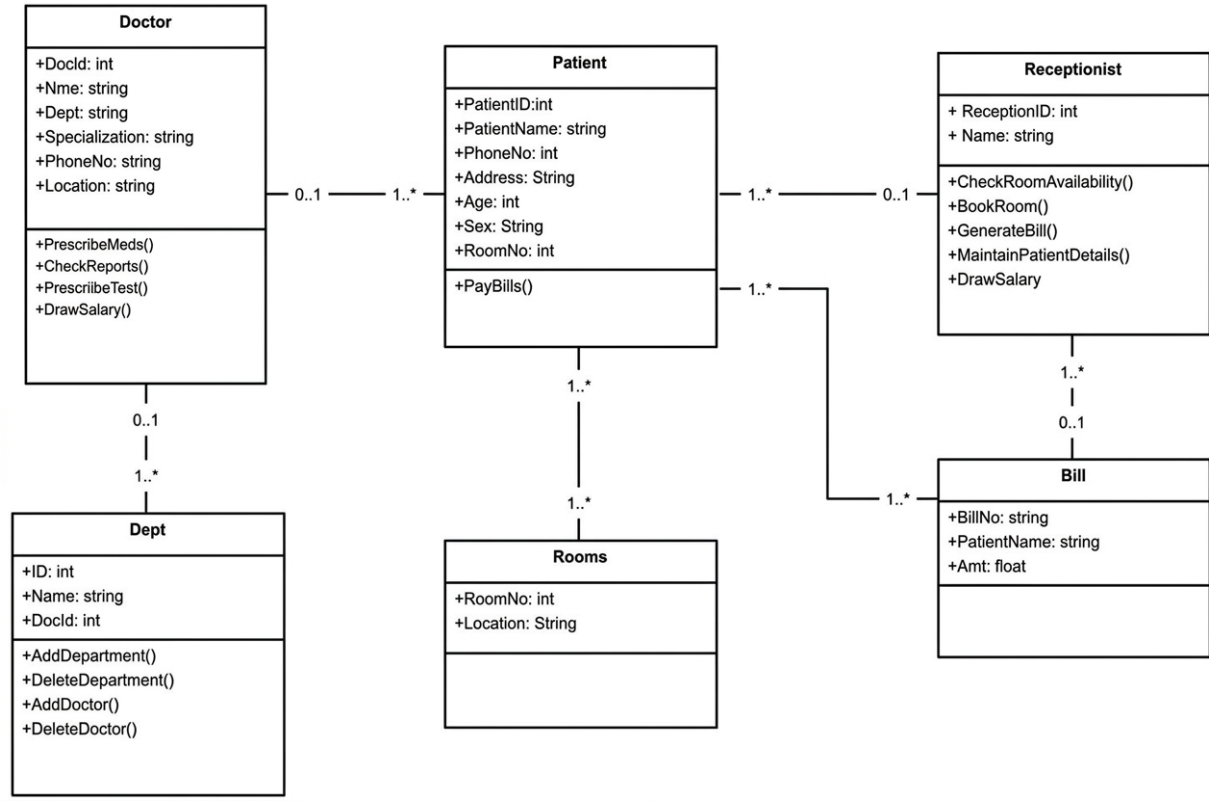
- **Analysis:** Symptom/Image Scanning.
- **Output:** Predictive Disease Scoring.



Hospital Management System with AI Diagnosis

4.iv. SCHEMA DESIGN (MongoDB)

Schema Diagram: Online Hospital Management System

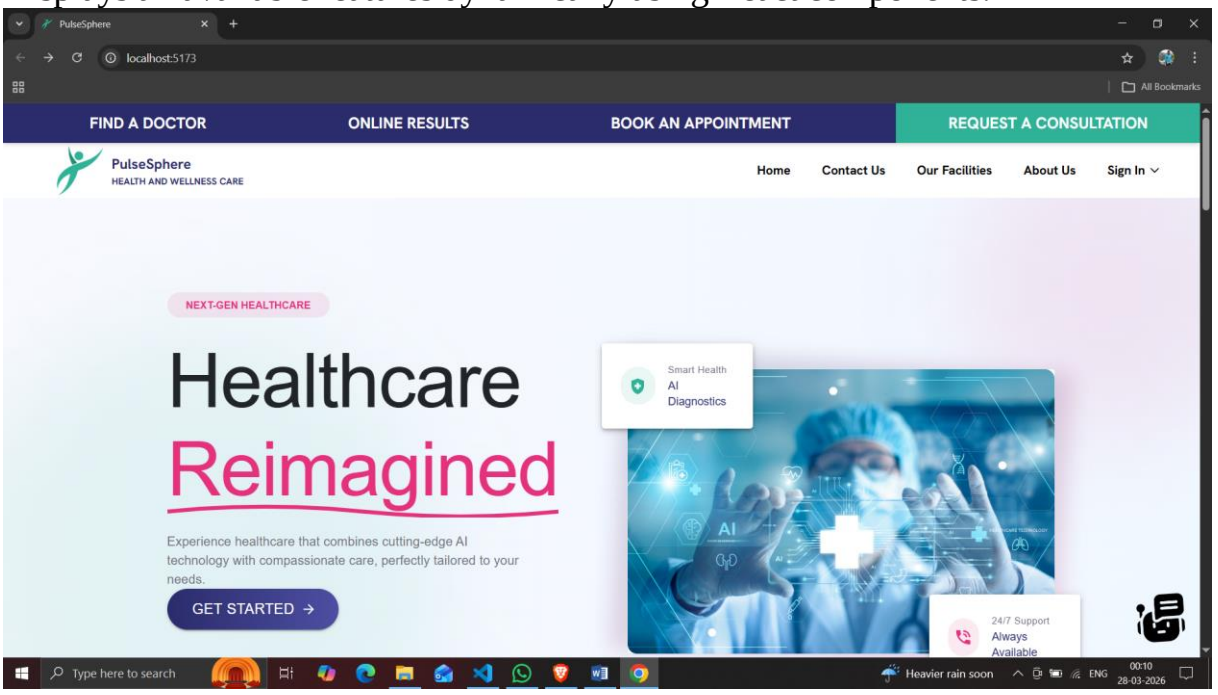


5. UI Codes

Frontend Codes

1. Home/Hero Page:

Displays all available features dynamically using React components.



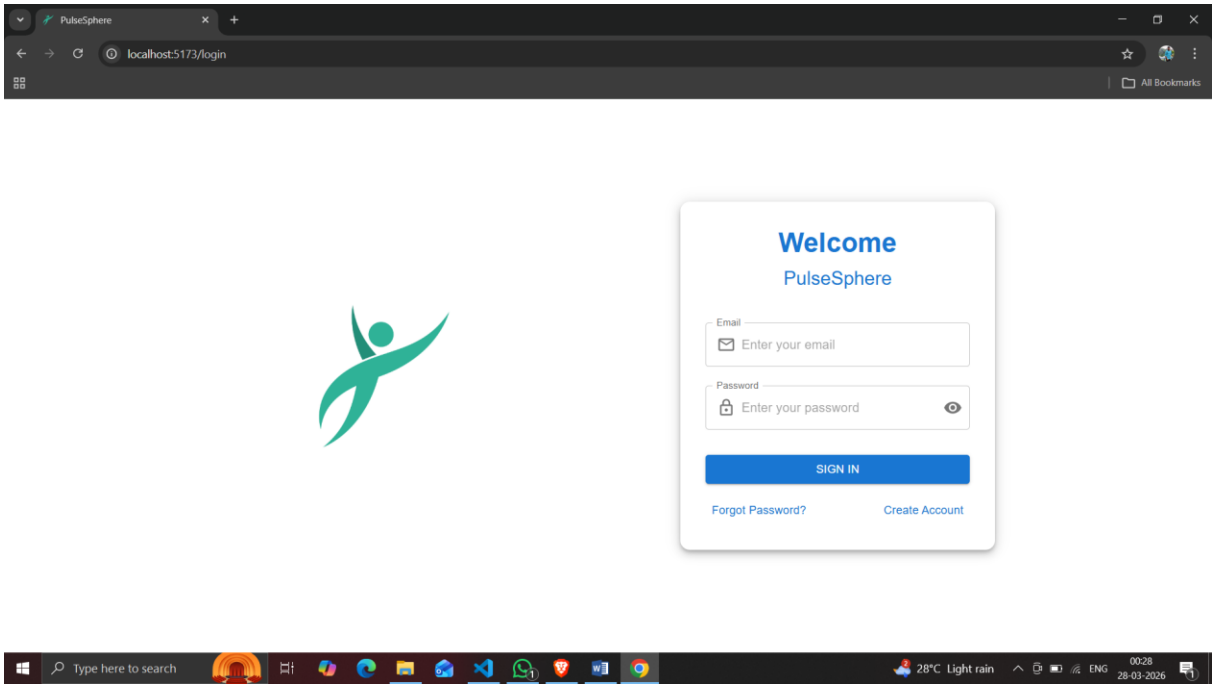
Code:

```
import React, { useState, useEffect } from "react";
import Nav from "../Nav Component/Nav";
import Footer from "../Nav Component/Footer";
import { Button, Container, Typography, Grid, Box, Card, CardContent } from
"@mui/material";
import { Star, PhoneInTalk, MedicalServices, People } from "@mui/icons-material";

function Home() {

  const [activeTestimonial, setActiveTestimonial] = useState(0);

  // Auto-rotate testimonials
  useEffect(() => {
    const interval = setInterval(() => {
      setActiveTestimonial((prev) => (prev + 1) % 3);
    }, 5000);
    return () => clearInterval(interval);
  }, []);

  const testimonials = [
```

```
  {
    quote: "The system has improved my healthcare experience.",
    name: "Sarah Johnson"
  },
  {
    quote: "Very helpful for managing my health records.",
    name: "Michael Chen"
  },
  {
    quote: "Easy to use and very secure platform.",
    name: "Elena Rodriguez"
  }
];

return (
  <div className="flex flex-col min-h-screen bg-white">

    {/* Navbar */}
    <Nav />

    {/* Hero Section */}
    <Container maxWidth="lg" className="py-16">
      <Typography variant="h3" className="font-bold mb-4">
        Healthcare Reimagined
      </Typography>

      <Typography variant="body1" className="mb-6">
        Smart healthcare system using AI for better patient care.
      </Typography>

      <Button variant="contained">
        Get Started
      </Button>
    </Container>

    {/* Stats Section */}
    <Container maxWidth="lg" className="py-10">
      <Grid container spacing={4}>
        {[
          { number: "98%", label: "Satisfaction", icon: <Star /> },
          { number: "24/7", label: "Support", icon: <PhoneInTalk /> },
          { number: "500+", label: "Doctors", icon: <MedicalServices /> },
          { number: "15k+", label: "Patients", icon: <People /> }
        ].map((item, index) => (
          <Grid item xs={6} md={3} key={index}>
            <Card>
              <CardContent>
                <Box>{item.icon}</Box>
                <Typography variant="h5">{item.number}</Typography>
                <Typography>{item.label}</Typography>
```

```
            </CardContent>
          </Card>
        </Grid>
      ))}
    </Grid>
  </Container>

  {/* Testimonials Section */}
  <Container maxWidth="lg" className="py-16">
    <Typography variant="h4" className="mb-6">
      Patient Feedback
    </Typography>

    <Card>
      <CardContent>
        <Typography variant="body1">
          "{testimonials[activeTestimonial].quote}"
        </Typography>
        <Typography variant="h6">
          - {testimonials[activeTestimonial].name}
        </Typography>
      </CardContent>
    </Card>
  </Container>

  {/* Footer */}
  <Footer />

 </div>
 );
}

export default Home;
```

2. Login Page:



A simple login form with email and password fields. If login is successful, users are redirected to their respective dashboards.

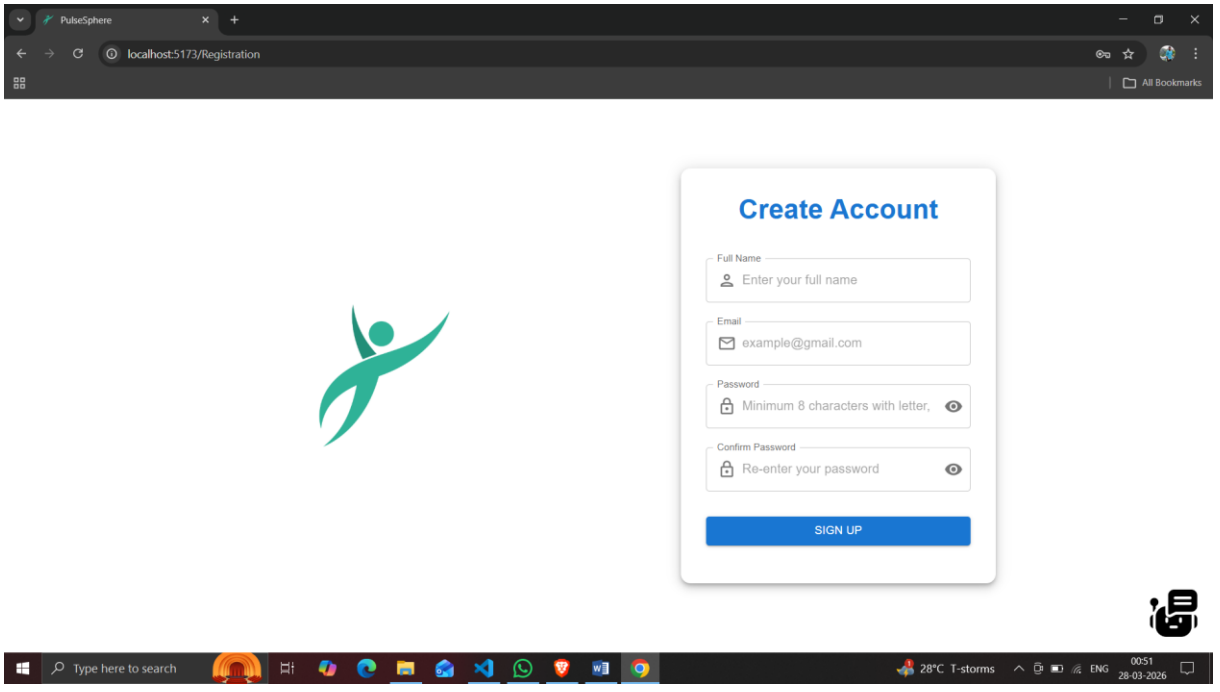
Code:

```
import React, { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import {
  Box, TextField, Button, Typography,
  CircularProgress, Container, Paper
} from "@mui/material";

function Login() {

  const [formData, setFormData] = useState({ email: "", password: "" });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const validateForm = () => {
    if (!formData.email || !formData.password) {
      setError("All fields are required");
      return false;
    }
    return true;
  };
```

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError("");

  if (!validateForm()) return;

  setLoading(true);
  try {
    const response = await axios.post(
      `${import.meta.env.VITE_API_URL}/api/auth/login`,
      formData
    );

    localStorage.setItem("token", response.data.token);

    // Redirect after login
    navigate("/");

  } catch (error) {
    setError("Login failed. Please check your credentials.");
  } finally {
    setLoading(false);
  }
};

return (
  <Box sx={{ height: "100vh", display: "flex", alignItems: "center" }}>
    <Container maxWidth="sm">
      <Paper sx={{ padding: 4 }}>

        <Typography variant="h4" align="center" gutterBottom>
          Login
        </Typography>

        {error && (
          <Typography color="error" align="center">
            {error}
          </Typography>
        )}

        <Box component="form" onSubmit={handleSubmit}>

          <TextField
            fullWidth
            margin="normal"
            label="Email"
            name="email"
            value={formData.email}
            onChange={handleChange}
```

```
        />

        <TextField
          fullWidth
          margin="normal"
          label="Password"
          type="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
        />

        <Button
          type="submit"
          fullWidth
          variant="contained"
          sx={{ mt: 2 }}
        >
          {loading ? <CircularProgress size={20} /> : "Login"}
        </Button>

      </Box>

    </Paper>
  </Container>
</Box>
```

3. Registration



;

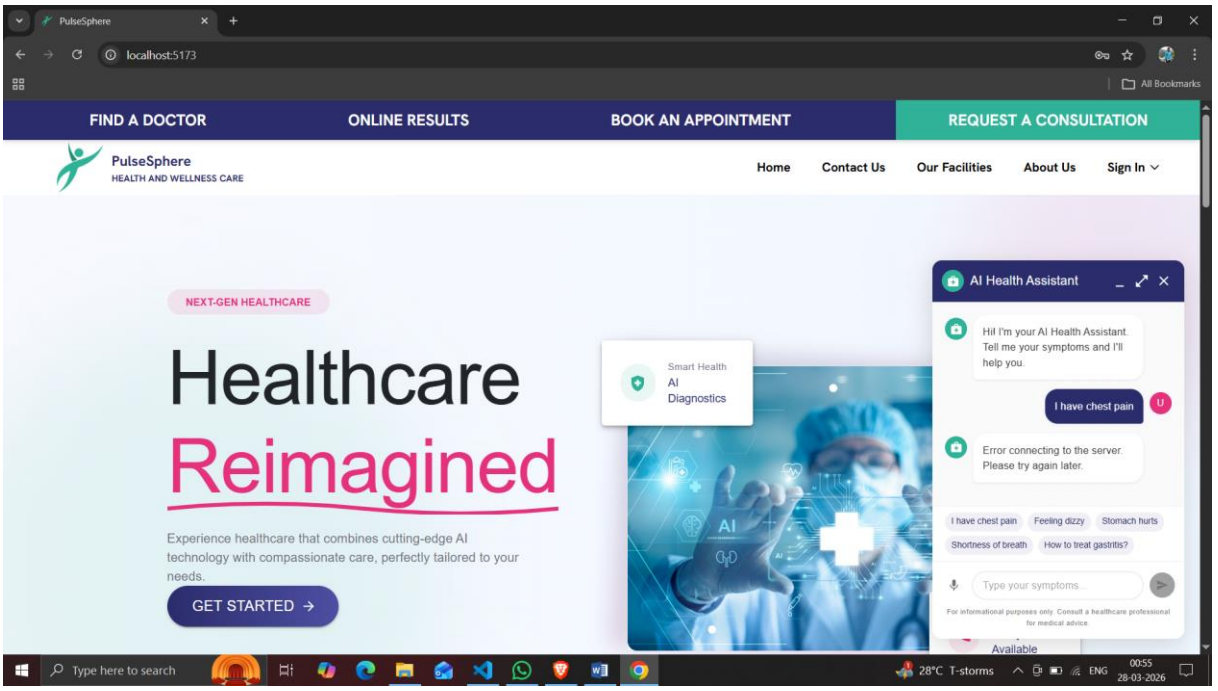
Code:

```
import React, { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import {
  Box, TextField, Button, Typography,
  CircularProgress, Container, Paper
} from "@mui/material";

function Registration() {

  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: ""
  });

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const validateForm = () => {
    if (!formData.name || !formData.email || !formData.password) {
      setError("All fields are required");
      return false;
    }
    return true;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");

    if (!validateForm()) return;

    setLoading(true);
    try {
```

```
    await axios.post(
      `${import.meta.env.VITE_API_URL}/api/auth/register`,
      formData
    );

    // Redirect after registration
    navigate("/login");

  } catch (error) {
    setError("Registration failed. Try again.");
  } finally {
    setLoading(false);
  }
};

return (
  <Box sx={{ height: "100vh", display: "flex", alignItems: "center" }}>
    <Container maxWidth="sm">
      <Paper sx={{ padding: 4 }}>

        <Typography variant="h4" align="center" gutterBottom>
          Register
        </Typography>

        {error && (
          <Typography color="error" align="center">
            {error}
          </Typography>
        )}

        <Box component="form" onSubmit={handleSubmit}>

          <TextField
            fullWidth
            margin="normal"
            label="Full Name"
            name="name"
            value={formData.name}
            onChange={handleChange}
          />

          <TextField
            fullWidth
            margin="normal"
            label="Email"
            name="email"
            value={formData.email}
            onChange={handleChange}
          />
```

```
            <TextField
              fullWidth
              margin="normal"
              label="Password"
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
            />

            <Button
              type="submit"
              fullWidth
              variant="contained"
              sx={{ mt: 2 }}
            >
              {loading ? <CircularProgress size={20} /> : "Sign Up"}
            </Button>

          </Box>

        </Paper>
      </Container>
    </Box>
  );
}

export default Registration;
;
```

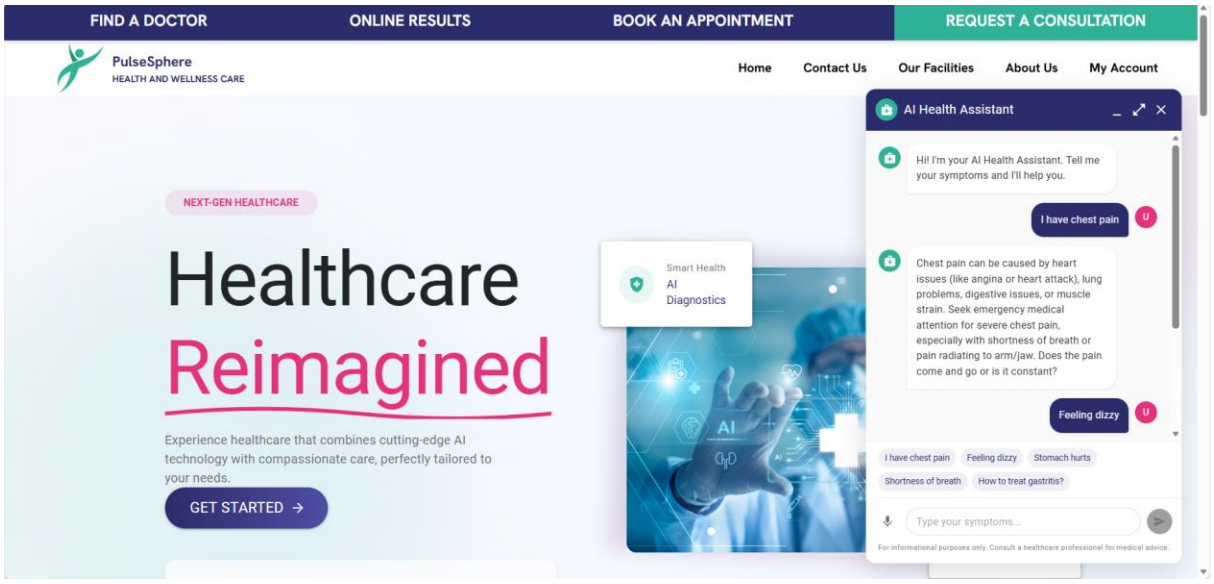
4. AI Assistance :



Code:

```
im{/* Smart Health Tools Section */}
<Grid container spacing={3}>

  {/* AI Symptom Analysis */}
  <Grid item xs={12} sm={6} md={3}>
    <Paper
      elevation={2}
      sx={{
        p: 3,
        textAlign: "center",
        cursor: "pointer",
        borderRadius: 2,
        "&:hover": { boxShadow: 4 }
      }}
      onClick={() => (window.location.href = "/symptom-analysis")}
    >
      <IconButton>
        <PsychologyIcon sx={{ color: "#e6317d" }} />
      </IconButton>

      <Typography variant="h6" fontWeight="bold">
        Symptom Analysis
      </Typography>

      <Typography variant="body2" color="text.secondary">
        AI-based system to analyze symptoms and suggest possible diseases.
      </Typography>
    </Paper>
  </Grid>
```

5. AI Tutor:



Code:

```
# main.py (FastAPI backend for AI Health Chatbot)
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import subprocess
import json
import pymongo
import re

app = FastAPI()

# CORS setup
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# MongoDB connection (update URI as needed)
client = pymongo.MongoClient("mongodb://localhost:27017")
db = client.healthcare
vitals_collection = db.vitals

class ChatRequest(BaseModel):
    message: str

# Health condition knowledge base
condition_info = {
    "heart_attack": {
```

```
        "symptoms": [
            "chest pain", "shortness of breath", "sweating",
            "dizziness", "jaw pain", "left arm pain", "nausea"
        ],
        "risk_factors": [
            "high blood pressure", "high cholesterol", "smoking",
            "diabetes", "obesity", "family history", "stress"
        ],
        "prevention": [
            "regular exercise", "healthy diet", "quit smoking",
            "limit alcohol", "manage stress", "regular check-ups"
        ],
        "emergency_signs": [
            "severe chest pain", "pain spreading to arms/jaw",
            "sudden shortness of breath", "cold sweat", "lightheadedness"
        ]
    },
    "gastritis": {
        "symptoms": [
            "stomach pain", "bloating", "heartburn", "nausea",
            "vomiting", "loss of appetite", "feeling full quickly"
        ],
        "risk_factors": [
            "h. pylori infection", "regular nsaid use", "excessive alcohol",
            "stress", "autoimmune disorders", "bile reflux"
        ],
        "prevention": [
            "avoid irritating foods", "limit alcohol", "eat smaller meals",
            "manage stress", "avoid nsaids", "treatment for h. pylori"
        ],
        "diet_recommendations": [
            "avoid spicy foods", "limit acidic foods", "avoid caffeine",
            "eat high-fiber foods", "stay hydrated", "eat regularly"
        ],
        "treatment": [
            "proton pump inhibitors", "acid reducers", "antacids",
            "antibiotics (for H. pylori)", "eliminate trigger foods",
            "stress reduction techniques", "smaller meals", "avoid alcohol"
        ]
    }
}

# Predefined responses for suggested messages
def get_suggested_response(message):
    """Provide predefined responses for common suggested queries"""
    message = message.lower().strip()

    # Map exact matches to concise responses (1-3 sentences)
    suggested_responses = {
        "i have chest pain": "Chest pain can be caused by heart issues (like angina or heart
```

attack), lung problems, digestive issues, or muscle strain. Seek emergency medical attention for severe chest pain, especially with shortness of breath or pain radiating to arm/jaw. Does the pain come and go or is it constant?",

"feeling dizzy": "Dizziness may be caused by inner ear problems, dehydration, blood pressure issues, or anxiety. For persistent dizziness or if accompanied by severe headache or vision changes, please seek medical attention. Are you experiencing any other symptoms with your dizziness?",

"stomach hurts": "Stomach pain could be indigestion, gastritis, food poisoning, or something more serious like ulcers or appendicitis. The location and timing of your pain can help determine the cause. Can you describe where exactly the pain is located?",

"shortness of breath": "Shortness of breath may result from respiratory issues, heart problems, anxiety, or overexertion. Sudden severe breathing difficulty, especially with chest pain, could be an emergency requiring immediate medical attention. Is this a new symptom for you?",

"how to treat gastritis?": "Gastritis treatment includes medications like antacids or acid reducers, avoiding trigger foods (spicy, acidic), eating smaller meals, and avoiding alcohol and NSAIDs. For persistent symptoms, please consult with your healthcare provider for proper diagnosis and treatment plan."
}

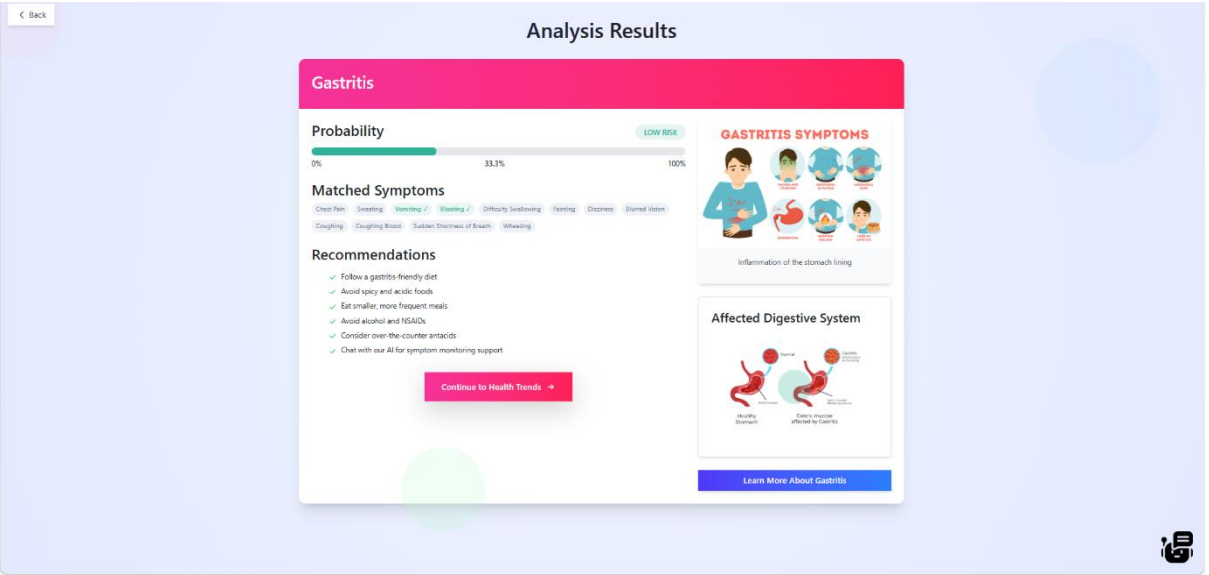
```
    return suggested_responses.get(message)

def get_condition_answer(message):
    """Extract health condition questions and provide answers based on knowledge base"""
    message = message.lower()

    # Check for general condition questions
    heart_attack_patterns = [
        r"heart attack", r"cardiac arrest", r"heart pain", r"heart condition"
    ]

    gastritis_patterns = [
        r"gastritis", r"stomach inflammation", r"stomach pain", r"acid reflux", r"indigestion"
    ]

    # Determine which condition is being asked about
    condition = None
    if any(re.search(pattern, message) for pattern in heart_attack_patterns):
        condition = "heart_attack"
    elif any(re.search(pattern, message) for pattern in gastritis_patterns):
        condition = "gastritis"

    if not condition:
        return None

    # Look for specific question types
```

```
    if re.search(r"symptom|sign|feel|experiencing", message):
        return f"Common symptoms of {condition.replace('_', ' ')} include: " + ",
".join(condition_info[condition]["symptoms"])

    elif re.search(r"cause|risk factor|reason", message):
        return f"Risk factors for {condition.replace('_', ' ')} include: " + ",
".join(condition_info[condition]["risk_factors"])

    elif re.search(r"prevent|avoid|stop", message):
        return f"Prevention measures for {condition.replace('_', ' ')} include: " + ",
".join(condition_info[condition]["prevention"])

    elif re.search(r"treat|cure|heal|therapy|medication", message) and condition == "gastritis":
        return f"Treatment options for gastritis include: " + ",
".join(condition_info[condition]["treatment"])

    elif condition == "heart_attack" and re.search(r"emergency|urgent|serious", message):
        return f"Emergency signs of a heart attack include: " + ",
".join(condition_info[condition]["emergency_signs"])

    elif condition == "gastritis" and re.search(r"diet|eat|food", message):
        return f"Dietary recommendations for gastritis: " + ",
".join(condition_info[condition]["diet_recommendations"])

    # General information about the condition
    elif condition == "heart_attack":
        return "A heart attack occurs when blood flow to part of the heart is blocked, causing
damage to heart muscle. It's a medical emergency requiring immediate attention. Common
symptoms include chest pain, shortness of breath, and pain radiating to the arm or jaw."

    elif condition == "gastritis":
        return "Gastritis is inflammation of the stomach lining, often caused by infection,
excessive alcohol, or regular use of certain pain relievers. Symptoms include stomach pain,
nausea, and reduced appetite. Most cases can be managed with lifestyle changes and
medication."

    return None

@app.post("/api/chat/analyze")
async def analyze_chat(request: ChatRequest):
    message = request.message

    # First, check if this is one of the suggested responses
    suggested_response = get_suggested_response(message)
    if suggested_response:
        return {"response": suggested_response}

    # Then, check if this is a specific condition question
    condition_answer = get_condition_answer(message.lower())
    if condition_answer:
```

```
        return {"response": condition_answer}

    # Very basic NLP to extract known symptoms from message
    message_lower = message.lower()
    known_symptoms = [
        "chest pain",
        "shortness of breath",
        "sweating",
        "dizziness",
        "jaw pain",
        "left arm pain",
        "nausea",
        "vomiting",
        "stomach pain",
        "bloating",
        "heartburn",
        "loss of appetite",
        "regurgitation",
        "difficulty swallowing",
        "racing heart",
        "coughing",
        "coughing blood",
    ]
    symptoms = [sym for sym in known_symptoms if sym in message_lower]

    if not symptoms:
        return {"response": "Please describe your symptoms in more detail, or ask a specific
question about heart attack or gastritis."}

    # Fetch latest vitals
    vitals = vitals_collection.find_one(sort=[("_id", -1)]) or {}
    bp = vitals.get("bp", 120)
    pulse = vitals.get("pulse", 75)
    sugar = vitals.get("sugar", 100)

    # Call Python analysis script
    try:
        process = subprocess.Popen(
            ["python", "model.py", json.dumps(symptoms)],
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
        )
        output, error = process.communicate()
        prediction = output.decode().strip()
        print("Prediction:", prediction)  # ✅ DEBUG LOG
    except Exception as e:
        print("Subprocess error:", e)
        return {"response": "Error running prediction engine."}

    # Modify severity based on vitals (cross-check)
```

```
    if "low" in prediction.lower():
        if bp > 140 or pulse > 100:
            prediction += (
                "\nNote: Your vitals indicate elevated risk. Consider medical advice."
            )

    return {
        "response": f"Based on symptoms ({', '.join(symptoms)}), the system suggests:
{prediction}"
    }

# New endpoint for structured symptom analysis
class SymptomAnalysisRequest(BaseModel):
    symptoms: list

@app.post("/api/novelty/analyze")
async def analyze_symptoms(request: SymptomAnalysisRequest):
    symptoms = [s.lower() for s in request.symptoms]

    # Call model.py with the symptoms for analysis
    try:
        process = subprocess.Popen(
            ["python", "model.py", json.dumps(symptoms)],
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
        )
        output, error = process.communicate()
        prediction = output.decode().strip()
        print("Structured Analysis Prediction:", prediction)
    except Exception as e:
        print("Subprocess error:", e)
        return {"prediction": "Error analyzing symptoms"}

    return {"prediction": prediction}

# Run this via: uvicorn main:app --reload --port 8000
```

6. Analysis Results:



Code:

```
import React, { useState } from "react";
import {
  Box,
  Typography,
  IconButton,
  Paper,
  Tooltip,
  Fade,
  Chip,
  Menu,
  MenuItem,
  ListItemIcon,
  ListItemText,
} from "@mui/material";
import {
  Delete as DeleteIcon,
  MoreVert as MoreIcon,
  PictureAsPdf as PdfIcon,
  Image as ImageIcon,
  InsertDriveFile as FileIcon,
  Download as DownloadIcon,
  Visibility as ViewIcon,
} from "@mui/icons-material";
```

```
import { deleteMedicalReport } from "../../../services/reportService";
import Swal from "sweetalert2";

function ReportItem({ report, onDelete }) {
  const [anchorEl, setAnchorEl] = useState(null);
  const [hover, setHover] = useState(false);
  const open = Boolean(anchorEl);

  const handleMenuClick = (event) => {
    setAnchorEl(event.currentTarget);
  };

  const handleMenuClose = () => {
    setAnchorEl(null);
  };

  const handleDelete = () => {
    handleMenuClose();
    Swal.fire({
      title: "Delete Report?",
      text: "This action cannot be undone.",
      icon: "warning",
      showCancelButton: true,
      confirmButtonColor: "#e03131",
      cancelButtonColor: "#6c757d",
      confirmButtonText: "Yes, delete it",
    }).then(async (result) => {
      if (result.isConfirmed) {
        try {
          await deleteMedicalReport(report._id);
          onDelete(report._id);
          Swal.fire("Deleted!", "Your report has been deleted.", "success");
        } catch (err) {
          console.error("Delete error:", err);
          Swal.fire("Error", "Failed to delete report.", "error");
        }
      }
    });
  };

  const fileName = report.fileName?.split("-").slice(1).join("-") || "report";
  const fileExtension = fileName.split(".").pop()?.toLowerCase() || "";

  // Determine file type icon
  const getFileIcon = () => {
    if (fileExtension === "pdf") {
      return <PdfIcon sx={{ color: "#e03131" }} />;
```

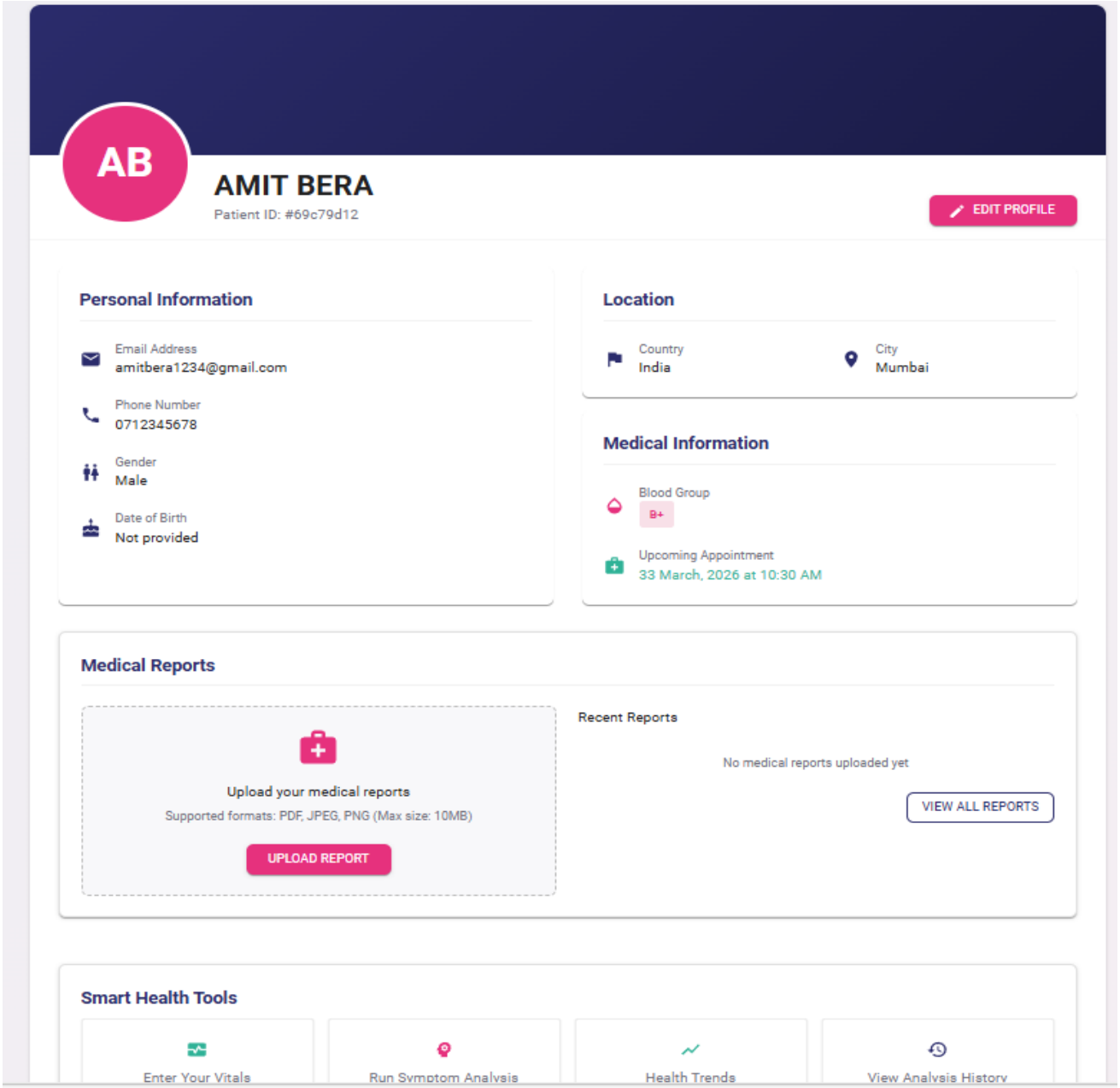
```
    } else if (["jpg", "jpeg", "png", "gif"].includes(fileExtension)) {
      return <ImageIcon sx={{ color: "#2fb297" }} />;
    } else {
      return <FileIcon sx={{ color: "#2b2c6c" }} />;
    }
  };

  // Format uploaded date
  const formattedDate = new Date(report.uploadedAt).toLocaleDateString("en-
US", {
    year: "numeric",
    month: "short",
    day: "numeric"
  });

  // Format file size if available
  const formatFileSize = (bytes) => {
    if (!bytes) return "";
    if (bytes < 1024) return bytes + " B";
    else if (bytes < 1048576) return (bytes / 1024).toFixed(1) + " KB";
    else return (bytes / 1048576).toFixed(1) + " MB";
  };

  return (
    <Paper
      elevation={hover ? 2 : 1}
      onMouseEnter={() => setHover(true)}
      onMouseLeave={() => setHover(false)}
      sx={{
        p: 2,
        mb: 2,
        display: "flex",
        alignItems: "center",
        justifyContent: "space-between",
        borderRadius: 2,
        transition: "all 0.3s ease",
        border: hover ? "1px solid #2fb297" : "1px solid #eaeaea",
        backgroundColor: hover ? "rgba(47, 178, 151, 0.03)" : "#ffffff",
        overflow: "hidden",
      }}
    >
      <Box sx={{ display: "flex", alignItems: "center", width: "100%" }}>
        <Box
          sx={{
            mr: 2,
            display: "flex",
            alignItems: "center",
```

```
          justifyContent: "center",
          width: 40,
          height: 40,
          borderRadius: 1,
          backgroundColor: "rgba(43, 44, 108, 0.05)"
        }}
      >
        {getFileIcon()}
      </Box>

      <Box sx={{ flex: 1 }}>
        <Typography fontWeight="medium" noWrap sx={{ maxWidth: "90%" }}>
          {fileName}
        </Typography>

        <Box sx={{ display: "flex", alignItems: "center", mt: 0.5 }}>
          <Typography variant="caption" color="text.secondary">
            {formattedDate}
          </Typography>

          <Fade in={hover}>
            <Chip
              size="small"
              label={fileExtension.toUpperCase()}
              sx={{
                ml: 1,
                fontSize: '0.65rem',
                height: 20,
                backgroundColor: fileExtension === "pdf" ? "rgba(224, 49, 49, 0.1)" :
                                 ["jpg", "jpeg", "png", "gif"].includes(fileExtension) ?
"rgba(47, 178, 151, 0.1)" :
                                 "rgba(43, 44, 108, 0.1)",
                color: fileExtension === "pdf" ? "#e03131" :
                       ["jpg", "jpeg", "png", "gif"].includes(fileExtension) ? "#2fb297" :
                       "#2b2c6c"
              }}
            />
          </Fade>

          {report.fileSize && (
            <Typography variant="caption" color="text.secondary" sx={{ ml: 1 }}>
              {formatFileSize(report.fileSize)}
            </Typography>
          )}
        </Box>
      </Box>
    </Box>
```

```
<Box>
  <Fade in={hover}>
    <Box sx={{ display: "flex" }}>
      <Tooltip title="Download">
        <IconButton
          href={`${import.meta.env.VITE_API_URL}/${report.filePath}`}
          target="_blank"
          download
          size="small"
          sx={{
            color: "#2fb297",
            "&:hover": {
              backgroundColor: "rgba(47, 178, 151, 0.1)",
            }
          }}
        >
          <DownloadIcon fontSize="small" />
        </IconButton>
      </Tooltip>
    </Box>
  </Fade>

  <IconButton
    onClick={handleMenuClick}
    size="small"
    sx={{
      color: hover ? "#2b2c6c" : "#71717a",
      "&:hover": { backgroundColor: "rgba(43, 44, 108, 0.1)" }
    }}
  >
    <MoreIcon fontSize="small" />
  </IconButton>

  <Menu
    anchorEl={anchorEl}
    open={open}
    onClose={handleMenuClose}
    transformOrigin={{ horizontal: 'right', vertical: 'top' }}
    anchorOrigin={{ horizontal: 'right', vertical: 'bottom' }}
    PaperProps={{
      elevation: 3,
      sx: { borderRadius: 2, minWidth: 175 }
    }}
  >
    <MenuItem
      component="a"
```

```
          href={`${import.meta.env.VITE_API_URL}/${report.filePath}`}
          target="_blank"
          download
        >
          <ListItemIcon>
            <DownloadIcon fontSize="small" sx={{ color: "#2fb297" }} />
          </ListItemIcon>
          <ListItemText>Download</ListItemText>
        </MenuItem>

        <MenuItem
          component="a"
          href={`${import.meta.env.VITE_API_URL}/${report.filePath}`}
          target="_blank"
        >
          <ListItemIcon>
            <ViewIcon fontSize="small" sx={{ color: "#2b2c6c" }} />
          </ListItemIcon>
          <ListItemText>View</ListItemText>
        </MenuItem>

        <MenuItem onClick={handleDelete}>
          <ListItemIcon>
            <DeleteIcon fontSize="small" sx={{ color: "#e03131" }} />
          </ListItemIcon>
          <ListItemText>Delete</ListItemText>
        </MenuItem>
      </Menu>
    </Box>
  </Paper>
 );
}

export default ReportItem;
```

6. Dashboard:



Code:

```
import React, { useState, useEffect } from "react";
import axios from "axios";
import {
  Box,
  Grid,
  Typography,
  Button,
  CircularProgress,
  Card,
  CardContent,
```

```
  Avatar,
  Divider,
  Paper,
  Chip,
  IconButton,
  useMediaQuery,
  useTheme,
} from "@mui/material";
import {
  Edit as EditIcon,
  Person as PersonIcon,
  Email as EmailIcon,
  Phone as PhoneIcon,
  Cake as CakeIcon,
  LocationOn as LocationIcon,
  Flag as CountryIcon,
  Wc as GenderIcon,
  Opacity as BloodIcon,
  MedicalServices as MedicalIcon,
  Logout as LogoutIcon,
  DeleteForever as DeleteIcon,
} from "@mui/icons-material";
import UpdatePatientProfile from "./UpdatePatientProfile";
import PsychologyIcon from "@mui/icons-material/Psychology";
import MonitorHeartIcon from "@mui/icons-material/MonitorHeart";
import HistoryIcon from "@mui/icons-material/History";
import ShowChartIcon from "@mui/icons-material/ShowChart";
import Swal from "sweetalert2";
import ReportUploadDialog from "../MedicalReports/ReportUploadDialog";
import ReportItem from "../MedicalReports/ReportItem";
import { getMedicalReports } from "../../../services/reportService";
import { useNavigate } from 'react-router-dom';

function PatientProfile() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [isEditing, setIsEditing] = useState(false);
  const [reports, setReports] = useState([]);
  const [loadingReports, setLoadingReports] = useState(false);
  const [isUploadDialogOpen, setIsUploadDialogOpen] = useState(false);
  const theme = useTheme();
  const isMobile = useMediaQuery(theme.breakpoints.down("sm"));
  const navigate = useNavigate();

  // Fetch user profile from API
  useEffect(() => {
    const fetchUser = async () => {
```

```
    try {
      const token = localStorage.getItem("token");
      const response = await axios.get(
        `${import.meta.env.VITE_API_URL}/api/users/profile`,
        {
          headers: { Authorization: `Bearer ${token}` },
        }
      );
      setUser(response.data);
    } catch (error) {
      console.error("Error fetching user data:", error);
    } finally {
      setLoading(false);
    }
  };

  fetchUser();
}, []);

// Fetch medical reports
useEffect(() => {
  const fetchReports = async () => {
    try {
      setLoadingReports(true);
      const data = await getMedicalReports();
      setReports(data);
    } catch (error) {
      console.error("Error fetching reports:", error);
    } finally {
      setLoadingReports(false);
    }
  };

  fetchReports();
}, []);

// Handle updated user data after edit
const handleProfileUpdate = (updatedUser) => {
  setUser(updatedUser);
  setIsEditing(false);
};

// Handle logout
const handleLogout = () => {
  Swal.fire({
    title: "Are you sure?",
    text: "You will be logged out of your account.",
```

```
    icon: "warning",
    showCancelButton: true,
    confirmButtonColor: "#2b2c6c",
    cancelButtonColor: "#d33",
    confirmButtonText: "Yes, log me out",
  }).then((result) => {
    if (result.isConfirmed) {
      localStorage.removeItem("token");
      Swal.fire({
        toast: true,
        position: "top-end",
        icon: "success",
        title: "✅ Logged out successfully",
        showConfirmButton: false,
        timer: 2500,
        timerProgressBar: true,
        background: "#f0f4ff",
        color: "#2b2c6c",
        iconColor: "#2fb297",
        customClass: {
          popup: "swal2-rounded",
        },
      });
      setTimeout(() => {
        window.location.href = "/";
      }, 2500);
    }
  });
};

// Handle account deletion
const handleDeleteAccount = () => {
  Swal.fire({
    title: "Are you sure?",
    text: "This action is permanent. Your account will be deleted.",
    icon: "warning",
    showCancelButton: true,
    confirmButtonColor: "#e03131",
    cancelButtonColor: "#2b2c6c",
    confirmButtonText: "Yes, delete my account",
  }).then((result) => {
    if (result.isConfirmed) {
      const token = localStorage.getItem("token");

      axios
        .delete(`${import.meta.env.VITE_API_URL}/api/users/delete`, {
          headers: { Authorization: `Bearer ${token}` },
```

```
        })
        .then(() => {
          localStorage.removeItem("token");
          Swal.fire({
            toast: true,
            position: "top-end",
            icon: "success",
            title: "🗑️ Account deleted successfully",
            showConfirmButton: false,
            timer: 2500,
            timerProgressBar: true,
            background: "#fff0f0",
            color: "#e03131",
            iconColor: "#e03131",
            customClass: {
              popup: "swal2-rounded",
            },
          });
          setTimeout(() => {
            window.location.href = "/login";
          }, 2500);
        })
        .catch((error) => {
          console.error("Error deleting account:", error);
          Swal.fire(
            "Error",
            "Failed to delete account. Please try again.",
            "error"
          );
        });
    }
  });
};

// Handle upload dialog
const handleOpenUploadDialog = () => {
  setIsUploadDialogOpen(true);
};

const handleCloseUploadDialog = () => {
  setIsUploadDialogOpen(false);
};

const handleUploadSuccess = async () => {
  // Refresh reports list
  try {
    const data = await getMedicalReports();
```

```
    setReports(data);
  } catch (error) {
    console.error("Error refreshing reports:", error);
  }
};

const handleDeleteReport = (reportId) => {
  setReports(reports.filter((report) => report._id !== reportId));
};

if (loading) {
  return (
    <Box
      sx={{
        display: "flex",
        justifyContent: "center",
        alignItems: "center",
        height: "80vh",
      }}
    >
      <CircularProgress sx={{ color: "#e6317d" }} />
    </Box>
  );
}

// Format date of birth
const formatDate = (dateString) => {
  if (!dateString) return "Not provided";

  const date = new Date(dateString);
  // Check if date is valid
  if (isNaN(date.getTime())) return "Not provided";

  return date.toLocaleDateString("en-US", {
    year: "numeric",
    month: "long",
    day: "numeric",
  });
};

// Calculate age from date of birth
const calculateAge = (dateString) => {
  if (!dateString) return "";

  const birthDate = new Date(dateString);
  // Check if date is valid and in the past
  if (isNaN(birthDate.getTime()) || birthDate > new Date()) return "";
```

```
  const today = new Date();
  let age = today.getFullYear() - birthDate.getFullYear();
  const monthDiff = today.getMonth() - birthDate.getMonth();

  if (
    monthDiff < 0 ||
    (monthDiff === 0 && today.getDate() < birthDate.getDate())
  ) {
    age--;
  }

  return age;
};

// Get initial letters for avatar
const getInitials = (name) => {
  return name
    .split(" ")
    .map((word) => word[0])
    .join("")
    .toUpperCase()
    .substring(0, 2);
};

return (
  <Box
    sx={{
      p: { xs: 2, md: 4 },
      minHeight: "100vh",
      bgcolor: "#f8f9fa",
      backgroundImage:
        "linear-gradient(135deg, rgba(43, 44, 108, 0.05) 0%, rgba(230, 49, 125, 0.05)
100%)",
    }}
  >
    <Card
      elevation={2}
      sx={{
        maxWidth: 1200,
        mx: "auto",
        mt: 2,
        borderRadius: 3,
        overflow: "visible",
        position: "relative",
        bgcolor: "#ffffff",
      }}
```

```
>
  {/* Header Banner */}
  <Box
    sx={{
      height: 180,
      bgcolor: "#2b2c6c",
      borderRadius: "12px 12px 0 0",
      backgroundImage:
        "linear-gradient(135deg, #2b2c6c 0%, #1a1b45 100%)",
      position: "relative",
    }}
  />

  {/* Profile Header with Avatar */}
  <Box
    sx={{
      px: { xs: 2, md: 4 },
      pt: 0,
      pb: 2,
      mt: -8,
      display: "flex",
      flexDirection: isMobile ? "column" : "row",
      alignItems: isMobile ? "center" : "flex-end",
      justifyContent: "space-between",
    }}
  >
    <Box
      sx={{
        display: "flex",
        flexDirection: isMobile ? "column" : "row",
        alignItems: isMobile ? "center" : "flex-end",
        mb: isMobile ? 2 : 0,
      }}
    >
      <Avatar
        sx={{
          width: 150,
          height: 150,
          border: "5px solid white",
          bgcolor: "#e6317d",
          fontSize: 48,
          fontWeight: "bold",
        }}
      >
        {getInitials(user.name)}
      </Avatar>
      <Box
```

```
      sx={{
        ml: isMobile ? 0 : 3,
        mt: isMobile ? 2 : 0,
        textAlign: isMobile ? "center" : "left",
      }}
    >
      <Typography variant="h4" fontWeight="bold">
        {user.name}
      </Typography>
      <Typography variant="subtitle1" color="#71717d">
        Patient ID: #{user._id?.substring(0, 8)}
      </Typography>
    </Box>
  </Box>

  <Button
    variant="contained"
    startIcon={<EditIcon />}
    onClick={() => setIsEditing(true)}
    sx={{
      bgcolor: "#e6317d",
      "&:hover": { bgcolor: "#c62a6a" },
      borderRadius: 2,
      px: 3,
    }}
  >
    Edit Profile
  </Button>
</Box>

<Divider />

{/* Profile Content */}
<CardContent sx={{ p: { xs: 2, md: 4 } }}>
  <Grid container spacing={4}>
    {/* Left Column - Personal Information */}
    <Grid item xs={12} md={6}>
      <Paper
        elevation={1}
        sx={{
          p: 3,
          borderRadius: 2,
          height: "100%",
          bgcolor: "#ffffff",
        }}
      >
        <Typography
```

```
  variant="h6"
  fontWeight="bold"
  color="#2b2c6c"
  gutterBottom
>
  Personal Information
</Typography>
<Divider sx={{ mb: 3, bgcolor: "#e6e6e6" }} />

<Grid container spacing={3}>
  <Grid item xs={12}>
    <Box sx={{ display: "flex", alignItems: "center" }}>
      <EmailIcon sx={{ color: "#2b2c6c", mr: 2 }} />
      <Box>
        <Typography variant="body2" color="#71717d">
          Email Address
        </Typography>
        <Typography variant="body1" fontWeight="medium">
          {user.email}
        </Typography>
      </Box>
    </Box>
  </Grid>

  <Grid item xs={12}>
    <Box sx={{ display: "flex", alignItems: "center" }}>
      <PhoneIcon sx={{ color: "#2b2c6c", mr: 2 }} />
      <Box>
        <Typography variant="body2" color="#71717d">
          Phone Number
        </Typography>
        <Typography variant="body1" fontWeight="medium">
          {user.mobile}
        </Typography>
      </Box>
    </Box>
  </Grid>

  <Grid item xs={12}>
    <Box sx={{ display: "flex", alignItems: "center" }}>
      <GenderIcon sx={{ color: "#2b2c6c", mr: 2 }} />
      <Box>
        <Typography variant="body2" color="#71717d">
          Gender
        </Typography>
        <Typography variant="body1" fontWeight="medium">
          {user.gender}
```

```
          </Typography>
        </Box>
      </Box>
    </Grid>

    <Grid item xs={12}>
      <Box sx={{ display: "flex", alignItems: "center" }}>
        <CakeIcon sx={{ color: "#2b2c6c", mr: 2 }} />
        <Box>
          <Typography variant="body2" color="#71717d">
            Date of Birth
          </Typography>
          <Box sx={{ display: "flex", alignItems: "center" }}>
            <Typography variant="body1" fontWeight="medium">
              {formatDate(user.dateOfBirth)}
            </Typography>
            {calculateAge(user.dateOfBirth) && (
              <Chip
                label={`${calculateAge(user.dateOfBirth)} years`}
                size="small"
                sx={{
                  ml: 2,
                  bgcolor: "#f0f0f0",
                  fontSize: "0.75rem",
                }}
              />
            )}
          </Box>
        </Box>
      </Box>
    </Grid>
  </Grid>
 </Paper>
</Grid>

{/* Right Column - Medical & Location */}
<Grid item xs={12} md={6}>
 <Grid container spacing={2} direction="column">
  {/* Location Info */}
  <Grid item>
    <Paper
      elevation={1}
      sx={{ p: 3, borderRadius: 2, bgcolor: "#ffffff" }}
    >
      <Typography
        variant="h6"
        fontWeight="bold"
```

```
      color="#2b2c6c"
      gutterBottom
    >
      Location
    </Typography>
    <Divider sx={{ mb: 3, bgcolor: "#e6e6e6" }} />

    <Grid container spacing={3}>
      <Grid item xs={12} sm={6}>
        <Box sx={{ display: "flex", alignItems: "center" }}>
          <CountryIcon sx={{ color: "#2b2c6c", mr: 2 }} />
          <Box>
            <Typography variant="body2" color="#71717d">
              Country
            </Typography>
            <Typography variant="body1" fontWeight="medium">
              {user.country}
            </Typography>
          </Box>
        </Box>
      </Grid>

      <Grid item xs={12} sm={6}>
        <Box sx={{ display: "flex", alignItems: "center" }}>
          <LocationIcon sx={{ color: "#2b2c6c", mr: 2 }} />
          <Box>
            <Typography variant="body2" color="#71717d">
              City
            </Typography>
            <Typography variant="body1" fontWeight="medium">
              {user.city}
            </Typography>
          </Box>
        </Box>
      </Grid>
    </Grid>
  </Paper>
</Grid>

{/* Medical Info */}
<Grid item>
  <Paper
    elevation={1}
    sx={{ p: 3, borderRadius: 2, bgcolor: "#ffffff" }}
  >
    <Typography
      variant="h6"
```

```
  fontWeight="bold"
  color="#2b2c6c"
  gutterBottom
>
  Medical Information
</Typography>
<Divider sx={{ mb: 3, bgcolor: "#e6e6e6" }} />

<Grid container spacing={3}>
  <Grid item xs={12}>
    <Box sx={{ display: "flex", alignItems: "center" }}>
      <BloodIcon sx={{ color: "#e6317d", mr: 2 }} />
      <Box>
        <Typography variant="body2" color="#71717d">
          Blood Group
        </Typography>
        <Box sx={{ display: "flex", alignItems: "center" }}>
          <Chip
            label={user.bloodGroup}
            sx={{
              bgcolor: "#f8e0e8",
              color: "#e6317d",
              fontWeight: "bold",
              borderRadius: 1,
            }}
          />
        </Box>
      </Box>
    </Box>
  </Grid>

  <Grid item xs={12}>
    <Box sx={{ display: "flex", alignItems: "center" }}>
      <MedicalIcon sx={{ color: "#2fb297", mr: 2 }} />
      <Box>
        <Typography variant="body2" color="#71717d">
          Upcoming Appointment
        </Typography>
        <Typography
          variant="body1"
          fontWeight="medium"
          color="#2fb297"
        >
          {new Date().getDate() + 5}{" "}
          {new Date().toLocaleString("default", {
            month: "long",
          })}
```

```
                , {new Date().getFullYear()} at 10:30 AM
              </Typography>
            </Box>
          </Box>
        </Grid>
      </Grid>
    </Paper>
  </Grid>
 </Grid>
</Grid>

{/* Medical Reports Section */}
<Grid item xs={12}>
 <Paper
  elevation={2}
  sx={{
    p: 3,
    borderRadius: 2,
    bgcolor: "#ffffff",
    border: `1px solid ${theme.palette.grey[300]}`,
    mb: 3,
  }}
 >
  <Typography
    variant="h6"
    fontWeight="bold"
    color="#2b2c6c"
    gutterBottom
  >
    Medical Reports
  </Typography>
  <Divider sx={{ mb: 3, bgcolor: "#e6e6e6" }} />

  <Grid container spacing={3}>
    {/* Upload Area */}
    <Grid item xs={12} md={6}>
      <Paper
        elevation={0}
        sx={{
          p: 3,
          borderRadius: 2,
          border: `1px dashed ${theme.palette.grey[400]}`,
          bgcolor: "rgba(43, 44, 108, 0.03)",
          height: "100%",
          display: "flex",
          flexDirection: "column",
          alignItems: "center",
```

```
      justifyContent: "center",
    }}
  >
    <MedicalIcon
      sx={{ fontSize: 48, color: "#e6317d", mb: 2 }}
    />
    <Typography
      variant="subtitle1"
      fontWeight="medium"
      textAlign="center"
      gutterBottom
    >
      Upload your medical reports
    </Typography>
    <Typography
      variant="body2"
      color="#71717d"
      textAlign="center"
      sx={{ mb: 3 }}
    >
      Supported formats: PDF, JPEG, PNG (Max size: 10MB)
    </Typography>
    <Button
      variant="contained"
      onClick={handleOpenUploadDialog}
      sx={{
        bgcolor: "#e6317d",
        "&:hover": { bgcolor: "#c62a6a" },
        borderRadius: 2,
        px: 3,
      }}
    >
      Upload Report
    </Button>
  </Paper>
</Grid>

{/* Recent Reports Area */}
<Grid item xs={12} md={6}>
  <Box sx={{ height: "100%" }}>
    <Typography
      variant="subtitle1"
      fontWeight="medium"
      gutterBottom
    >
      Recent Reports
    </Typography>
```

```
{loadingReports ? (
  <Box
    sx={{
      display: "flex",
      justifyContent: "center",
      alignItems: "center",
      height: "100%",
    }}
  >
    <CircularProgress sx={{ color: "#e6317d" }} />
  </Box>
) : reports.length === 0 ? (
  <Box
    sx={{
      display: "flex",
      flexDirection: "column",
      alignItems: "center",
      mt: 4,
      mb: 2,
    }}
  >
    <Typography
      variant="body2"
      color="#71717d"
      textAlign="center"
    >
      No medical reports uploaded yet
    </Typography>
  </Box>
) : (
  <Box sx={{ mt: 2 }}>
    {reports.map((report) => (
      <ReportItem
        key={report._id}
        report={report}
        onDelete={handleDeleteReport}
      />
    ))}
  </Box>
)}

<Box
  sx={{
    display: "flex",
    justifyContent: "flex-end",
    mt: 3,
```

```
          }}
        >
          <Button
            variant="outlined"

            sx={{
              borderColor: "#2b2c6c",
              color: "#2b2c6c",
              "&:hover": {
                borderColor: "#1e1f4b",
                bgcolor: "rgba(43, 44, 108, 0.04)",
              },
              borderRadius: 2,
            }}
          >
            View All Reports
          </Button>
        </Box>
      </Box>
    </Grid>
  </Grid>
 </Paper>
</Grid>

{/* Smart Health Tools Section */}
<Grid item xs={12}>
 <Paper
  elevation={2}
  sx={{
    p: 3,
    borderRadius: 2,
    bgcolor: "#ffffff",
    border: `1px solid ${theme.palette.grey[300]}`,
  }}
 >
  <Typography
    variant="h6"
    fontWeight="bold"
    color="#2b2c6c"
    gutterBottom
  >
    Smart Health Tools
  </Typography>

  <Grid container spacing={2}>
    {[
      {
```

```
    label: "Enter Your Vitals",
    icon: <MonitorHeartIcon sx={{ color: "#2fb297" }} />,
    path: "/enter-vitals",
  },
  {
    label: "Run Symptom Analysis",
    icon: <PsychologyIcon sx={{ color: "#e6317d" }} />,
    path: "/symptom-analysis",
  },

  {
    label: "Health Trends",
    icon: <ShowChartIcon sx={{ color: "#2fb297" }} />,
    path: "/health-trends",
  },
  {
    label: "View Analysis History",
    icon: <HistoryIcon sx={{ color: "#2b2c6c" }} />,
    path: "/analysis-history",
  },
].map(({ label, icon, path }) => (
  <Grid item xs={12} sm={6} md={3} key={label}>
    <Paper
      elevation={1}
      sx={{
        p: 2,
        textAlign: "center",
        cursor: "pointer",
        border: `1px solid ${theme.palette.grey[200]}`,
        "&:hover": {
          boxShadow: 4,
          borderColor: "#828487",
        },
      }}
      onClick={() => (window.location.href = path)}
    >
      <IconButton>{icon}</IconButton>
      <Typography
        variant="subtitle1"
        color="#71717d"
        fontWeight="medium"
      >
        {label}
      </Typography>
    </Paper>
  </Grid>
))}
```

```
    </Grid>
  </Paper>
</Grid>

{/* Account Actions Section */}
<Grid item xs={12}>
  <Paper
    elevation={1}
    sx={{ p: 3, borderRadius: 2, bgcolor: "#ffffff" }}
  >
    <Typography
      variant="h6"
      fontWeight="bold"
      color="#2b2c6c"
      gutterBottom
    >
      Account Actions
    </Typography>
    <Divider sx={{ mb: 3, bgcolor: "#e6e6e6" }} />
    <Box
      sx={{
        display: "flex",
        justifyContent: "flex-end",
        gap: 2,
        flexWrap: "wrap",
      }}
    >
      <Button
        variant="outlined"
        startIcon={<LogoutIcon />}
        onClick={handleLogout}
        sx={{
          borderColor: "#2b2c6c",
          color: "#2b2c6c",
          "&:hover": {
            borderColor: "#1e1f4b",
            bgcolor: "rgba(43, 44, 108, 0.04)",
          },
          borderRadius: 2,
        }}
      >
        Log Out
      </Button>
      <Button
        variant="contained"
        startIcon={<DeleteIcon />}
        onClick={handleDeleteAccount}
```

```
                sx={{
                  bgcolor: "#FF0000",
                  "&:hover": { bgcolor: "#d32f2f" },
                  borderRadius: 2,
                }}
              >
                Delete My Account
              </Button>
            </Box>
          </Paper>
        </Grid>
      </Grid>
    </CardContent>
  </Card>

  {/* Update Profile Modal */}
  {isEditing && (
    <UpdatePatientProfile
      user={user}
      onClose={() => setIsEditing(false)}
      onUpdate={handleProfileUpdate}
    />
  )}

  {/* Report Upload Dialog */}
  {isUploadDialogOpen && (
    <ReportUploadDialog
      onClose={handleCloseUploadDialog}
      onSuccess={handleUploadSuccess}
    />
  )}
</Box>
);
}

export default PatientProfile;
```

Backend API Endpoints (Express.js)

- **POST /api/auth/register**: Registers new patients, doctors, or administrators.

- **POST /api/auth/login**: Authenticates users and returns a secure JWT token.

- **POST /api/appointments/book**: Allows patients to schedule a consultation slot with a doctor.

- **GET /api/appointments/:userId**: Retrieves all upcoming and past appointments for a specific user.

- **POST /api/ai/diagnose**: Sends symptom data and medical images to the AI engine for analysis.

- **GET /api/medical-records/:patientId**: Fetches the Electronic Health Records (EHR) and history for a patient.

- **PUT /api/medical-records/update**: Allows doctors to add consultation notes and final diagnoses to a record.

➢ DB.js

```
import mongoose from "mongoose";

/**
 * Establishing connection to MongoDB Atlas (Cloud)
 * This handles the persistence of patient records and AI logs.
 */
const connectDB = async () => {
  try {
    // Connect to the database using the URI stored in environment variables
    await mongoose.connect(process.env.MONGO_URI);

    console.log("Cloud Database Connection Established: MongoDB
Connected");
  } catch (error) {
    // Log any connection errors and terminate the process
    console.error(`Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;
```

➢ Course_model.js

```
import mongoose from "mongoose";

// Define AI Diagnosis Schema separately for cleaner nesting (Similar to
lessonSchema)
const aiDiagnosisSchema = new mongoose.Schema({
  conditionDetected: { type: String, required: true },
  confidenceScore: { type: Number, required: true }, // e.g., 0.95 for 95%
  aiAnalysisSummary: { type: String },
  recommendedNextSteps: [String]
});

// Define Consultation Schema (Similar to sectionSchema)
const consultationSchema = new mongoose.Schema({
  doctorName: { type: String, required: true },
  visitDate: { type: Date, default: Date.now },
  symptomsReported: [String],
  aiResults: [aiDiagnosisSchema] // Nested AI findings for this specific visit
});

// Define Main Patient Record Schema (Similar to courseSchema)
const patientRecordSchema = new mongoose.Schema({
  patientName: {
    type: String,
    required: true,
    trim: true
  },
  patientId: {
    type: String,
    required: true,
    unique: true
  },
  age: {
    type: Number,
    required: true
  },
  gender: {
    type: String,
    enum: ["Male", "Female", "Other"],
    required: true
  },
  bloodGroup: {
    type: String,
    enum: ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"]
  },
  medicalHistory: [String], // Previous surgeries, allergies, etc.
  contactNumber: {
    type: String,
    required: true
```

```
  },
  emergencyContact: {
    type: String
  },
  admissionStatus: {
    type: String,
    enum: ["Outpatient", "Inpatient", "Discharged"],
    default: "Outpatient"
  },
  image: {
    type: String // URL to profile picture or identity scan
  },
  consultations: [consultationSchema] // Nested array of all doctor visits
}, { timestamps: true });

const PatientRecord = mongoose.model("PatientRecord", patientRecordSchema);

export default PatientRecord;;
```

➢ Enrollment.js

```
import mongoose from "mongoose";

const appointmentSchema = new mongoose.Schema({

  patientId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User" // References the Patient user
  },

  doctorId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User" // References the Doctor user
  },

  consultationStatus: {
    type: String,
    enum: ["Scheduled", "In-Progress", "Completed", "Cancelled"],
    default: "Scheduled"
  },

  appointmentDate: {
    type: Date,
    required: true
  }

});

export default mongoose.model("Appointment", appointmentSchema);));
```

➢ Progress.js:

```
    import mongoose from "mongoose";

const recoveryProgressSchema = new mongoose.Schema({

  patientId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  },

  diagnosisId: {
    type: String, // Linking to the specific AI Diagnosis session
    required: true
  },
```

```
  symptomUpdates: [String], // Array of daily health updates or symptom changes

  recoveryPercentage: {
    type: Number, // Calculated based on vital improvements (0 to 100)
    default: 0
  }

});

export default mongoose.model("RecoveryProgress", recoveryProgressSchema);
    ;
```

➢ Auth_route:

```
iimport express from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import User from "../models/User.js";

const router = express.Router();

// REGISTER - Create Account for Patient, Doctor, or Admin
router.post("/register", async (req, res) => {
  try {
    const { name, email, password, role } = req.body;

    // Validation
    if (!name || !email || !password || !role) {
      return res.status(400).json({
        success: false,
        message: "All fields (name, email, password, role) are required"
      });
    }

    const existingUser = await User.findOne({ email });

    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: "User already exists with this email"
      });
    }

    // Password Hashing
    const hashedPassword = await bcrypt.hash(password, 10);
```

```
    const user = new User({
      name,
      email,
      password: hashedPassword,
      role // Assigning hospital role: Patient, Doctor, or Admin
    });

    await user.save();

    res.status(201).json({
      success: true,
      message: "Medical account registered successfully"
    });

  } catch (error) {
    console.error(error);
    res.status(500).json({
      success: false,
      message: "Server error during registration"
    });
  }
});

// LOGIN - Authenticate Medical Staff or Patient
router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        success: false,
        message: "Email and password required"
      });
    }

    const user = await User.findOne({ email });

    if (!user) {
      return res.status(400).json({
        success: false,
        message: "Medical account not found"
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(400).json({
        success: false,
        message: "Invalid medical credentials"
```

```
      });
    }

    // Generate Secure JWT Token
    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "1d" }
    );

    res.status(200).json({
      success: true,
      message: "Login successful",
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
      }
    });

  } catch (error) {
    console.error(error);
    res.status(500).json({
      success: false,
      message: "Server error during login"
    });
  }
});

export default router;

;
```

➢ progress_route:

```
import express from "express";
import RecoveryProgress from "../models/RecoveryProgress.js";
import PatientRecord from "../models/PatientRecord.js";

const router = express.Router();

// SAVE RECOVERY PROGRESS
// This replaces the "Save Progress" logic to track patient health milestones
router.post("/", async (req, res) => {
  try {
    const { patientId, diagnosisId, milestoneId } = req.body;

    // Find existing recovery progress for this specific diagnosis
```

```
    let recovery = await RecoveryProgress.findOne({ patientId, diagnosisId });

    // Fetch the patient record to determine total treatment milestones required
    const patient = await PatientRecord.findOne({ patientId: patientId });

    // Logic: Calculate progress based on completed milestones vs total required steps
    // Defaulting to 1 to avoid division by zero
    const totalMilestones = patient?.requiredTreatments?.length || 5;

    if (!recovery) {
      // Create new recovery tracking if it doesn't exist
      recovery = new RecoveryProgress({
        patientId,
        diagnosisId,
        completedMilestones: [milestoneId],
        recoveryPercentage: 0
      });
    } else {
      // Check if this medical milestone was already logged
      const alreadyLogged = recovery.completedMilestones.some(
        (id) => id.toString() === milestoneId
      );

      if (!alreadyLogged) {
        recovery.completedMilestones.push(milestoneId);
      }
    }

    // Update the recovery percentage (e.g., 2 out of 5 steps completed = 40%)
    recovery.recoveryPercentage =
      (recovery.completedMilestones.length / totalMilestones) * 100;

    await recovery.save();

    res.json({
      success: true,
      message: "Patient recovery status updated",
      recovery
    });

  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Failed to update recovery progress" });
  }
});

// GET RECOVERY STATUS
// This allows Doctors and Patients to view the healing journey
router.get("/:patientId/:diagnosisId", async (req, res) => {
  try {
```

```
    const recovery = await RecoveryProgress.findOne({
      patientId: req.params.patientId,
      diagnosisId: req.params.diagnosisId
    });

    res.json(recovery || { recoveryPercentage: 0, completedMilestones: [] });

  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Failed to fetch patient recovery status" });
  }
});

export default router;r;
```

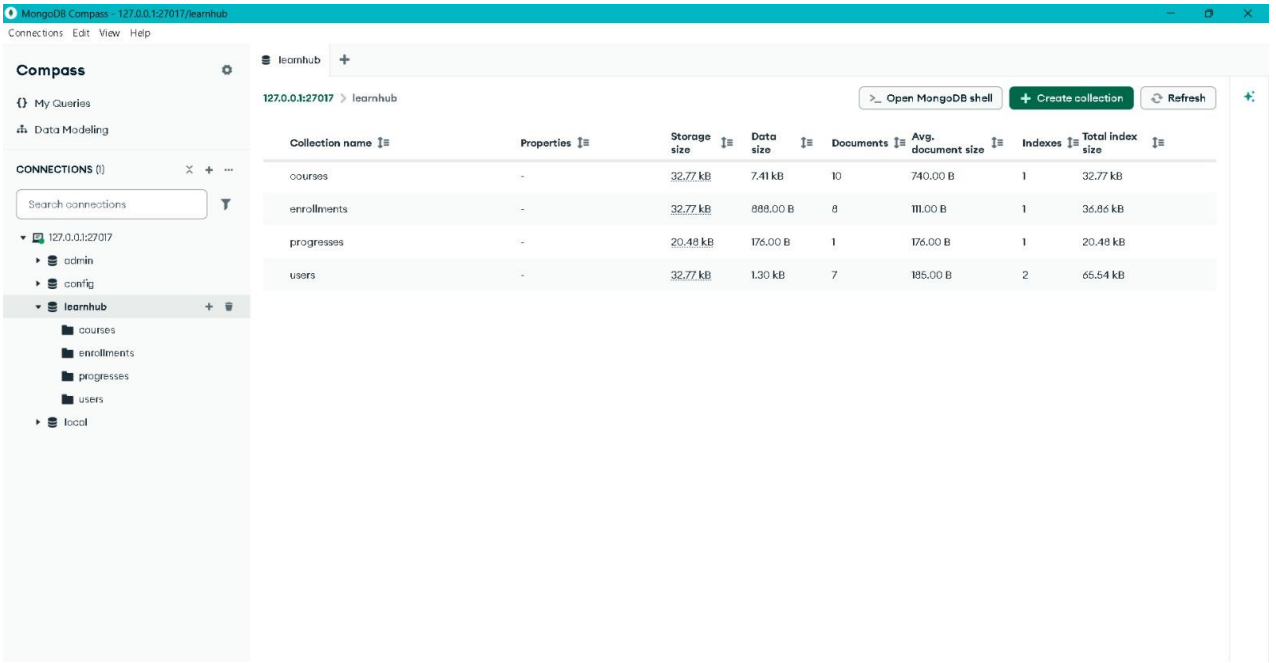
➢ Server.js:

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import connectDB from "./config/db.js";

/* Route Imports */
import authRoutes from "./routes/auth.js";
import patientRoutes from "./routes/patients.js"; // Replaces courseRoutes
import aiRoutes from "./routes/ai.js";
import recoveryRoutes from "./routes/recovery.js"; // Replaces progressRoutes
import appointmentRoutes from "./routes/appointments.js"; // Replaces
enrollmentRoutes

dotenv.config();
const app = express();

/* Middleware & DB */
app.use(cors());
app.use(express.json());
connectDB();

/* Base Route */
app.get("/", (req, res) => res.send("Hospital Management API is running"));

/* Routes */
app.use("/api/auth", authRoutes);
app.use("/api/patients", patientRoutes);
app.use("/api/ai", aiRoutes);
app.use("/api/recovery", recoveryRoutes);
app.use("/api/appointments", appointmentRoutes);
```

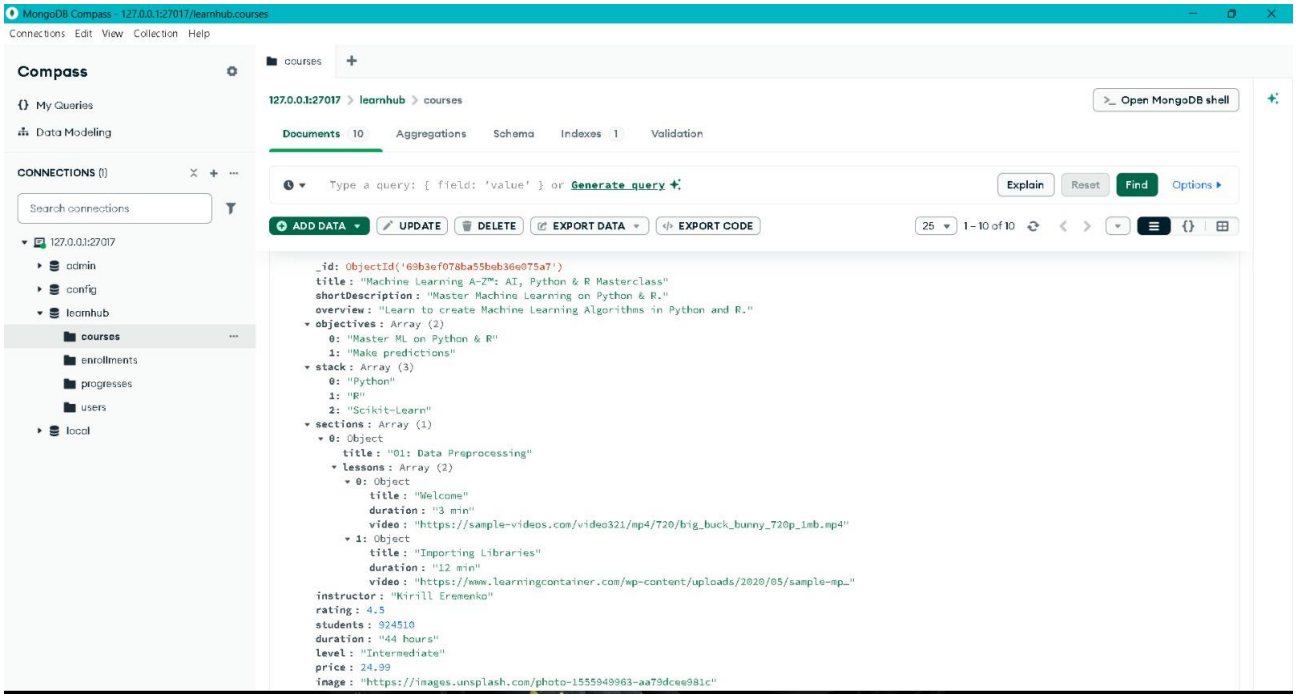
```
/* Server Listen */
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`🚀 HMS Server running on port ${PORT}`)););
```

DataBase(Mongodb):

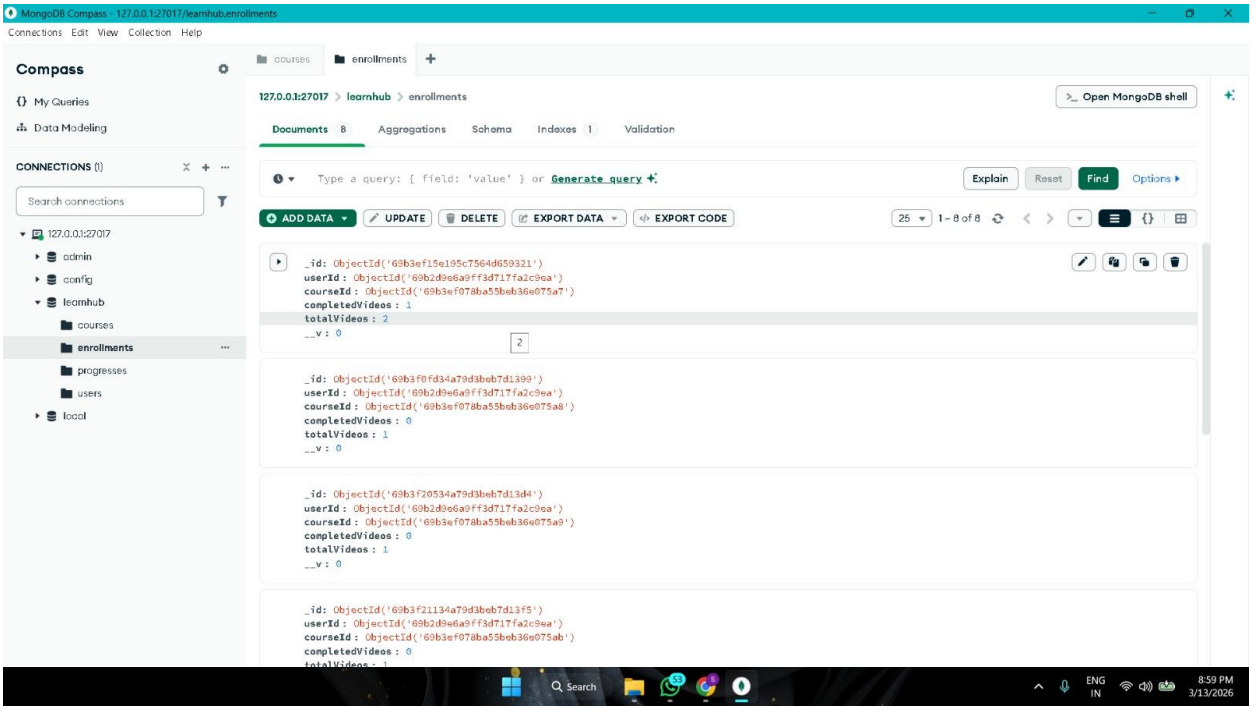
❖Cluster:



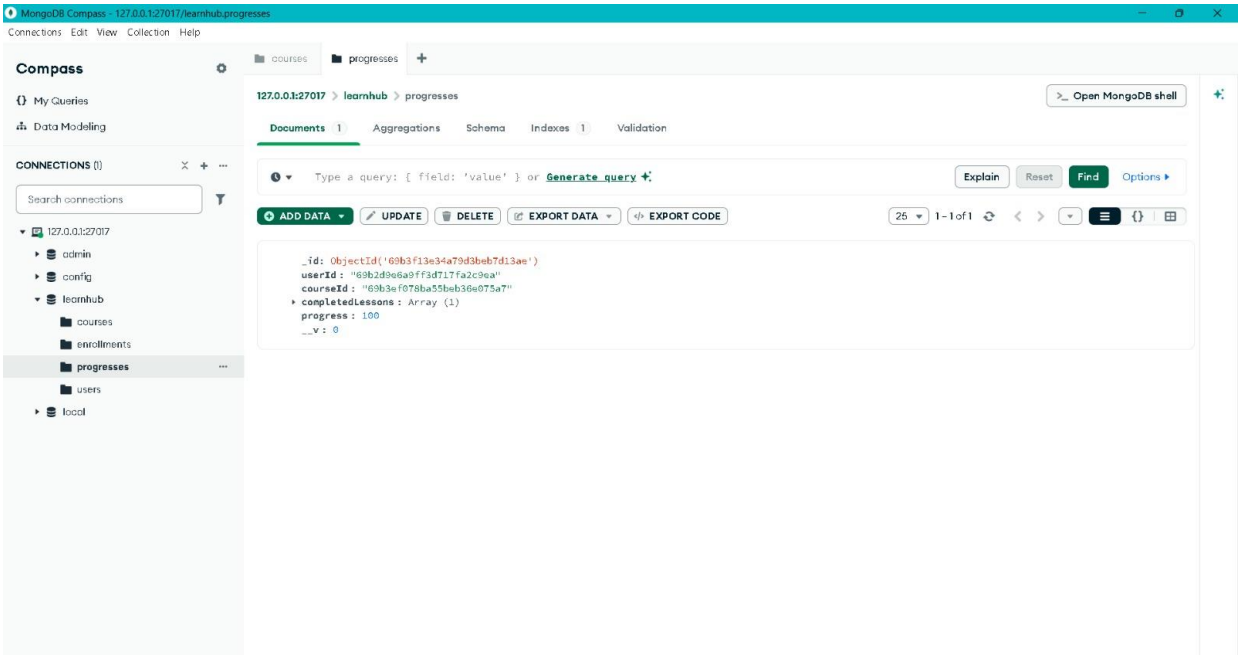
❖ Collections:



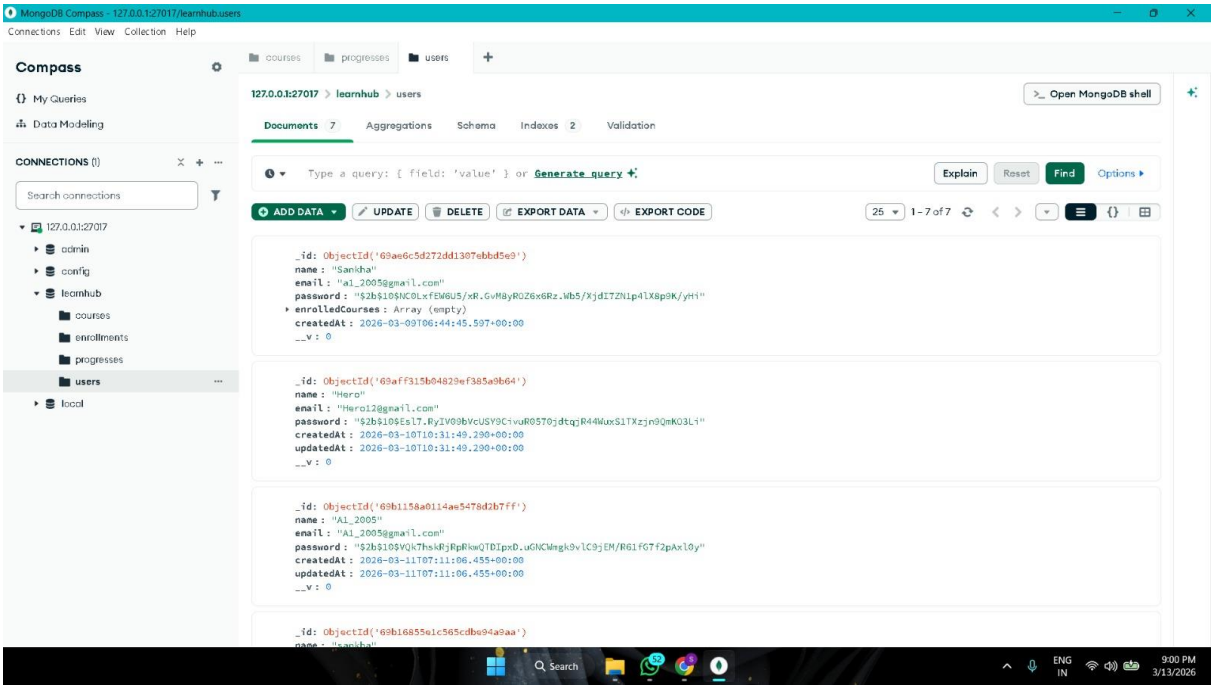
❖Enrolment Collection:



❖Progress Collection



❖User Collection



6. CONCLUSION

8.i Summary of Achievement

The development of the **Cloud-Based Hospital Management System (HMS) with AI Diagnosis** represents a successful synthesis of modern full-stack engineering and healthcare technology. By utilizing the **MERN stack** and **Cloud infrastructure**, this project has demonstrated a robust, secure, and scalable architecture capable of handling sensitive patient data, real-time appointment scheduling, and complex AI-driven medical analysis. The implementation effectively transitions from traditional, fragmented hospital record-keeping to a unified, intelligent digital ecosystem, bridging the gap between clinical efficiency and patient-centric care.

8.ii Key Project Outcomes

- **Seamless Clinical Experience:** The platform provides an intuitive, browser-based interface for both doctors and patients, removing administrative barriers and allowing medical professionals to focus entirely on patient treatment.

- **AI-Driven Diagnostic Insights:** Through the integration of the **AI Diagnosis Engine**, the system offers doctors preliminary, data-backed assessments. This marks a significant advancement over traditional portals, providing on-demand analysis that assists in early disease detection.

- **Data-Driven Recovery Tracking:** By implementing the **Recovery Progress** and **Appointment** modules, the system offers patients clear, quantified visibility into their healing journey, improving treatment adherence and transparency.

- **Secure & Scalable Architecture:** The choice of a document-oriented database (**MongoDB**) and a modular backend ensures the platform can securely store massive amounts of medical imagery and records while remaining extensible for future healthcare features.

7. FUTURE SCOPE & FURTHER ENHANCEMENTS

Future Scope:

9.i. Advanced Medical Modules

- **Telemedicine:** Real-time WebRTC video consultations.
- **Lab Sync:** Automated integration with Laboratory Information Systems.
- **E-Prescriptions:** PDF generation for digitally signed prescriptions.

9.ii. AI & Data Intelligence

- **Predictive Care:** ML models to identify chronic disease risks.
- **Advanced Imaging:** Deep Learning (CNN) for X-ray and MRI analysis.

9.iii. Platform Scalability & Ecosystem

- **Payments:** Integrated billing and insurance claim APIs.
- **IoT Integration:** Real-time vitals syncing from wearable devices.
- **Mobile App:** Cross-platform portability via React Native.
- **Cloud Optimization:** Containerization using Docker and Kubernetes.

8. BIBLIOGRAPHY

- **https://www.mongodb.com** – Official documentation for MongoDB Atlas cloud database management.
- **https://www.reactjs.org** – Documentation for building the interactive Patient and Doctor dashboards.
- **https://nodejs.org** – Reference for the asynchronous backend runtime environment.
- **https://expressjs.com** – Documentation for designing the RESTful APIs for medical data handling.
- **https://aws.amazon.com/health** – AWS healthcare cloud compliance and hosting standards.
- **https://developer.mozilla.org** – MDN Web Docs for standardized web development practices.
- **https://github.com** – Version control and repository management for the project source code.
- **Software Engineering: A Practitioner's Approach (Pressman, R. S.)** – Reference for the Waterfall Model and System Development Life Cycle (SDLC) methodologies